

Sustainable multi-objective re-entrant hybrid flow shop scheduling with batch-dependent setup via a Meta-reinforcement learning guided algorithm

Abstract

Optimizing multiple objectives in re-entrant manufacturing systems has become increasingly important in modern production environments. A sustainable multi-objective scheduling problem for a re-entrant hybrid flow shop with batch-dependent setups is investigated in this study, aiming to simultaneously optimize makespan, total energy consumption cost, and carbon trading cost. First, a mixed-integer linear programming model incorporating four machine operating states is formulated to accurately characterize the problem. Subsequently, a Meta-Q-learning guided generalized variable neighborhood search (GVNS)-embedded NSGA-II is proposed to tackle this complex scheduling problem. In particular, a Meta-Q-learning driven GVNS framework is introduced to adaptively select neighborhood operators based on search feedback, enhancing the balance between exploration and exploitation. Meanwhile, several problem-specific mechanisms are developed to enhance algorithmic performance, including a two-phase decoding scheme, a hierarchical hybrid initialization strategy, and a hybrid decision-objective space crowding distance measure for maintaining population diversity. Moreover, a resurrection-based reinitialization strategy is employed to avoid premature convergence and enhance robustness. Extensive computational experiments show that the proposed algorithm performs markedly better than several existing methods in both solution convergence and distribution, particularly for large-scale instances. The proposed approach provides an effective optimization tool for sustainable and intelligent manufacturing scheduling.

Keywords: Re-entrant hybrid flow shop; Meta-reinforcement learning; multi-objective optimization; sustainable scheduling; batch-dependent setup.

1. Introduction

Hybrid flow shop scheduling problems (HFSPs) have been widely investigated because they can effectively model complex manufacturing systems with multiple processing stages and parallel machines at each stage. Such systems are commonly encountered in semiconductor manufacturing, steel production, and other capital-intensive industries, where production efficiency and resource utilization are critical performance concerns (Ribas et al.,2010). HFSPs are well known to be NP-hard, and their computational complexity increases rapidly when realistic production constraints are incorporated (Neufeld et al.,2023; Ruiz & Vázquez-Rodríguez,2010). In many practical manufacturing environments,

jobs may revisit one or more processing stages multiple times due to technological requirements, reworks, or repeated treatments, resulting in the re-entrant hybrid flow shop scheduling problem (RHFSP). Re-entrant characteristics are common in semiconductor fabrication and surface treatment processes, where repeated coating, etching, or annealing operations are required (Wu & Xie,2023). Compared to classical HFSPs, RHFSPs introduce more complex precedence relationships and job interactions. These characteristics significantly enlarge the solution space and complicating the scheduling task (Wu & Cao,2022). Consequently, traditional flow shop assumptions are often insufficient to accurately represent such realistic production systems.

In addition, modern manufacturing systems are subject to a variety of practical constraints. These include sequence-dependent setup times (SDST) (Lu et al.,2025), batch-processing machines, and batch-related setup requirements that arise when machines switch between different batches or product families (Shahvari & Logendran,2016). Ignoring such operational characteristics may lead to schedules that are infeasible or inefficient when implemented in real production settings. Accordingly, increasing research efforts have focused on extending HFSP and RHFSP models to incorporate these practical production features.

With the growing complexity of production systems, sustainability considerations have become a major concern in manufacturing planning and control (Dong & Ye,2022). In hybrid flow shop environments, energy consumption is closely related to processing decisions, setup operations, batch formation, machine idle states, and transportation activities (Zhang et al.,2024a). Additionally, the introduction of time-of-use (TOU) electricity pricing and carbon-related policies further complicates the scheduling problem, naturally leading to multi-objective optimization formulations that balance production efficiency with energy and environmental performance (Li et al.,2023).

To address the conflicting objectives arising in such complex scheduling environments, multi-objective evolutionary algorithms (MOEAs) have been extensively adopted. Classical Pareto-based approaches, such as Non-dominated Sorting Genetic Algorithm II (NSGA-II), have demonstrated strong performance in maintaining solution diversity and handling conflicting objectives (Deb et al.,2002). More recently, decomposition-based frameworks have gained increasing attention. As they are effective in handling large-scale and complex multi-objective scheduling problems. (Jiang et al.,2021). Despite these methodological advances, traditional evolutionary and metaheuristic algorithms typically rely on fixed search operators and static parameter settings. Their performance may degrade when facing highly complex, dynamic, or large-scale scheduling environments. To overcome these limitations, learning-enhanced solution approaches have attracted growing interest in recent years. Reinforcement learning (RL), particularly Q-learning and deep reinforcement learning (DRL), has been increasingly integrated into scheduling algorithms to enable adaptive decision-making (Tao et al.,2024). Typical applications include learning-based selection of neighborhood structures, control of local search operators, and adaptive guidance of evolutionary search processes (Song et al.,2025). More advanced

learning-based techniques have also been explored, including imitation learning, transfer learning, and Meta-reinforcement learning. These approaches aim to leverage prior experience and historical solutions to improve adaptability under complex and changing scheduling environments (Shao et al.,2024; Zhang et al.,2024b).

Nevertheless, several research gaps remain. First, most of the existing studies focus either on practical production constraints or learning-based solution strategies, with relatively few studies jointly considering re-entrant hybrid flow shop structures, green and energy-related objectives, and adaptive learning mechanisms within a unified multi-objective optimization framework. Moreover, although batch-related setups are common in batch-oriented production environments, the batch-dependent setup (BDST) constraint has rarely been investigated in RHFSPs, particularly under sustainable multi-objective settings. Second, only limited research simultaneously considers three important objectives, namely makespan, total energy consumption cost (TEC), and carbon trading cost (CTC). Lastly, while Meta-reinforcement learning methods have shown promising results, their integration with MOEAs in the RHFSP environment remains limited, especially the adaptive operator selection driven by reinforcement learning algorithms, which has been scarcely explored in RHFSPs.

To fill these gaps, a sustainable multi-objective RHFSP with batch-dependent setup (SMO-RHFSP-BDST) is investigated. A Meta-Q-learning-based MOEA, termed MQNSGA-II, is proposed to solve the problem. The main contributions of this paper are as follows.

- A novel SMO-RHFSP-BDST problem is formally formulated by integrating re-entrant processing, batch-dependent setup constraints, and a multi-objective optimization model that incorporating four machine operating states is established to minimize the makespan, TEC and CTC simultaneously.
- A MQNSGA-II algorithm is proposed, in which Meta-Q-learning (MQL) is employed to drive a generalized variable neighborhood search (GVNS) embedded within the NSGA-II framework. Combined with a hierarchical hybrid initialization strategy, thereby adaptively guiding operator and neighborhood selection and balancing global exploration and local exploitation.
- A hybrid crowding distance mechanism that combines information from both the decision and objective spaces is designed to improve the distribution of the Pareto solution set.
- A resurrection-based reinitialization strategy is introduced to mitigate search stagnation and premature convergence. It helps improve the diversity and stability of the solution set.

The rest of the paper is arranged as follows. Section 2 reviews related works; Section 3 formalizes the problem and establishes the mathematical model; Section 4 provides a detailed description of the proposed MQNSGA-II algorithm; Section 5 presents numerical experiments and result analysis; Section 6 concludes the paper and discusses future research directions.

2. Literature review

2.1 HFS scheduling and re-entrant variants

HFSP has received wide attention from both academia and industry. This is because it can better describe modern manufacturing systems with multi-stage parallel processing. Early studies mainly focused on the standard HFSP. Most of them concentrated on modeling and optimization with classical objectives such as minimizing the makespan (Ribas et al., 2010). However, these studies are usually based on simplified assumptions and may not fully capture the operational complexity of real production environments. To improve the realism of scheduling models, later research introduced various practical manufacturing features. One important extension is the RHFSP, in which jobs may revisit one or more processing stages multiple times (Yan & Wu, 2022). Wu and Xie (2023) noted that, compared with non-re-entrant HFSPs, RHFSPs are more difficult due to repeated resource contention at bottleneck stages and complex synchronization among jobs. Several studies have investigated RHFSPs under different modeling assumptions. Lin et al. (2020) examined an RHFSP with limited buffer capacity and demonstrated the effectiveness of their solution approach. Ren et al. (2021) analyzed the impact of stage revisits on scheduling performance by studying RHFSP without hybrid structures. Wu and Cao (2022) further investigated hybrid configurations with parallel machines and proposed metaheuristic methods to cope with the enlarged search space. In addition, some studies have considered more integrated decision frameworks. For example, Wu et al. (2022) incorporated job release decisions into RHFSP models, and showed that jointly optimizing release and scheduling can improve system performance compared with decoupled decision-making strategies. Overall, existing studies indicate that re-entrant hybrid flow shop scheduling has evolved from simplified structural models to more integrated and system-oriented formulations (Wu & Liu, 2025).

2.2 Practical production constraints and green scheduling considerations

To enhance the realism of HFSP and RHFSP models, a wide range of practical production constraints have been incorporated into scheduling formulations (Chen et al., 2025b). SDST is widely recognized as a critical constraint in HFS and related scheduling problems because the setup duration depends on the processing order of jobs. Yunusoglu and Topaloglu Yildiz (2021) investigated an unrelated parallel machine scheduling problem with SDST, Zheng et al. (2024) studied a RHFSP with SDST and limited buffers. More recently, Wang et al. (2026) introduced group setup times associated with product family transitions, highlighting the importance of modeling setup operations when switching between heterogeneous product groups in practical manufacturing systems. Batch-processing machines represent another important class of constraints. Batch formation decisions interact strongly with sequencing and machine assignment. Improper batching may lead to excessive waiting times or inefficient resource utilization (Huang et al., 2016). Batch-processing HFSPs have been studied under both productivity-driven and energy-aware objectives. These studies highlight

the trade-off between processing efficiency and operational cost (Costa,2015; Wu et al.,2021; Zhang et al.,2024a). In many practical batch-oriented production environments, setup requirements may depend on the transition between different batches rather than solely on the sequence of individual jobs. Shahvari and Logendran (2016) demonstrated that BDST can better represent practical production scenarios involving product families or rework groups compared with conventional SDST.

In addition, green and sustainable scheduling has attracted growing attention in HFSP. Luo et al. (2013) investigated HFSP considering machine electricity consumption cost and Mansouri et al. (2016) analyzed the trade-off between production efficiency and energy consumption in flow shop scheduling. Ding et al. (2016) introduced carbon-efficient flow shop scheduling models that jointly minimize makespan and carbon emissions through multi-objective optimization techniques. Later research incorporated more comprehensive energy modeling in hybrid flow shop environments (Lu et al.,2022). Jiang et al. (2021) investigated energy-efficient distributed hybrid flow shop scheduling with multiprocessor tasks and Ding et al. (2021) investigated flexible flow shop scheduling with TOU electricity tariffs. Under TOU pricing schemes, scheduling decisions become time-coupled because shifting operations to off-peak periods may reduce electricity costs but potentially increase completion times (Wang et al.,2023).

Overall, the integration of above practical production constraints has transformed HFSP scheduling into a typical multi-objective optimization problem with strong complexity and trade-off characteristics. Consequently, MOEAs have attracted increasing attention as a suitable solution framework for such problems.

2.3 MOEAs for Scheduling

MOEAs have been widely used to scheduling problems due to their population-based search mechanism and their ability to approximate the Pareto front in a single run (Sharma & Kumar,2022). Among various MOEAs, the NSGA-II proposed by Deb et al. (2002) is one of the most influential algorithms. It employs fast non-dominated sorting and a crowding-distance mechanism to maintain diversity among solutions. Wang et al. (2023) extended NSGA-II by incorporating adaptive genetic operators and historical information strategies to address an energy-efficient fuzzy HFSP. Li et al. (2023) proposed a Q-learning (QL) and GVNS driven NSGA-II to enhance exploration and exploitation during the search process. Furthermore, NSGA-II has been applied to other complex manufacturing scheduling problems. Shao et al. (2025) developed an adaptive job scheduling NSGA-II algorithm for a flexible job shop scheduling problem with variable sublots and operator constraints. In addition, NSGA-II has also been used as a benchmark algorithm for evaluating new optimization methods. Zheng et al. (2026) proposed CHIO-DECM algorithm was compared with several classical algorithms, including NSGA-II, to verify its effectiveness. Overall, these studies demonstrate that NSGA-II provides a robust and effective framework for solving multi-objective scheduling problems.

2.4 Adaptive and Learning-based Scheduling Approaches

To overcome the limited adaptability of conventional metaheuristics with fixed operators and static parameter settings, recent studies have increasingly explored learning-based scheduling methods (Drake et al.,2020). Building on this trend, Lin et al. (2024) applied DRL to RHFSP, and showed that DRL can directly learn machine- and job-selection policies in automated material handling systems. Li et al. (2024) proposed a hybrid graph-based imitation learning method, which improved generalization by transferring expert knowledge instead of learning from scratch. Li et al. (2025) combined a feature-driven double deep Q-network with iterated greedy for RHFSP, thereby enhancing adaptive decision-making during the scheduling process. Wu and Liu (2025) demonstrated the potential of DRL for adaptive scheduling in dynamic and multi-objective environments. Furthermore, Wang et al. (2025) designed a heterogeneous graph self-attention DRL framework, while Chen et al. (2025a) further combined evolutionary multitask optimization with genetic programming, indicating that learning-based scheduling is moving toward more realistic, dynamic, and multi-objective production settings.

To improve robustness and reduce training dependence, another important research direction incorporates RL into metaheuristic frameworks as an adaptive control mechanism. Karimi-Mamaghan et al. (2022) integrated QL into iterated greedy and showed that learning can guide operator choice more effectively than fixed control. Wang and Wang (2022) incorporated RL into a cooperative memetic algorithm for HFSP and used an offline-trained policy agent to adaptively guide operator selection during the search process. Ren et al. (2023) used QL to select local search operators in variable neighborhood iterative search. Li et al. (2023) integrated QL into a multi-objective metaheuristic for energy-efficient HFSP and used RL to guide adaptive operator selection. Tao et al. (2024) incorporated RL into an iterated greedy algorithm to adaptively select neighborhood operators during local search and thus improve search efficiency in the rescheduling. Lu et al. (2024) combined four metaheuristics with QL for lot-streaming HFSP with consistent sublots. Zhang et al. (2024b) proposed a Meta-RL framework for HFSP, in which a Meta-training phase was used to acquire prior operator knowledge and an online QL phase was designed to adapt operator selection during search. Shao et al. (2024) extended this idea by developing a Meta-QL-based multi-objective metaheuristic and improved the transferability of learned operator-selection knowledge across instances. Yu et al. (2024) further applied QL-based metaheuristics to dynamic multi-objective surgery scheduling and verified the effectiveness of RL in guiding local search strategy selection during rescheduling. Overall, these studies show that RL is increasingly used as an adaptive search controller rather than a direct scheduling policy, making metaheuristics more responsive and scalable for complex HFSPs.

Table 1 summarizes existing studies on RHFSP from problem focus, objectives, and methods. Existing works mainly focus on efficiency or energy objectives, and recent studies introduce learning-based approaches to improve adaptability.

Nevertheless, effectively integrating learning mechanisms with multi-objective evolutionary frameworks in RHFS environments, while maintaining convergence, diversity, and robustness across heterogeneous instances, remains a research challenge.

Table 1 Literature analysis.

Problem focus	Objectives	Methods	References
RHFSP with batch processing machines	$C_{\max}$	Heuristic construction methods	Huang et al.,2016
RHFSP, green scheduling	$C_{\max}$, TEC and total carbon emissions	Improved hybrid salp swarm, NSGA-III	Dong & Ye,2022
RHFSP with batch processing machines	$C_{\max}$, TEC	Improved MOEA based on decomposition (IMOEAD)	Wu & Cao,2022
RHFSP	$C_{\max}$, TEC	IMOEAD based on inverse optimization and decomposition	Wu et al.,2022
RHFSP	Output deviation, total waiting time	IMOEAD	Yan & Wu,2022
RHFSP	$C_{\max}$, transferring cost	IMOEAD with local search	Wu & Xie,2023
HFSP with reworking	$C_{\max}$	Iterated Greedy, QL	Tao et al.,2024
RHFSP	$C_{\max}$	DRL	Lin et al.,2024
RHFSP with SDST	Weighted $C_{\max}$	Cooperative adaptive genetic algorithm	Zheng et al.,2024
HFSP with reworkS	Total tardiness	DRL	Hyeon-II et al.,2025
RHFSP	$C_{\max}$	Deep Q-network, Iterated Greedy	Li et al.,2025
RHFSP, green scheduling	$C_{\max}$, TEC and average total machine workload standard deviations	Modified multi-agent proximal policy optimization algorithm	Wu & Liu,2025
RHFSP, green scheduling	$C_{\max}$, total carbon emissions	Improved Coronavirus Herd Immunity Optimizer	Zheng et al.,2026
Sustainable RHFSP with BDST	$C_{\max}$, TEC and CTC	Meta-QL, GVNS, NSGA-II	Our

3. Problem Description and Modeling

3.1. Problem Description

The SMO-RHFSP-BDST is described as follows. A set of n jobs must be processed sequentially through k stages. At stage $j \in \{1, 2, \dots, k\}$, there are m_j parallel unrelated machines, each characterized by a specific processing speed coefficient v_{jm} and a power coefficient λ_{jm} , where $m \in \{1, 2, \dots, m_j\}$ denotes the machine index at stage j . During the initial processing phase, all jobs are processed in the order of stages $1 \rightarrow 2 \rightarrow \dots \rightarrow k$, the final stage k corresponds to a quality inspection operation. Jobs that fail inspection must undergo rework. For each job i the rework starting stage is denoted by $r_i \in \{1, 2, \dots, k-1\}$, indicating the stage from which job i must restart processing. The rework route follows the same stage order as the initial processing, i.e., from stage r_i to stage k constituting a complete rework flow. After all jobs complete the initial processing and inspection, rework jobs are grouped into batches according to their rework starting stages. Jobs sharing the same rework starting stage are assigned to the same logical rework batch B_s , where

$s \in \{1, 2, \dots, b\}$ denotes the batch index and b is the total number of batches. Formally, $B_s = \{i | r_i = s, i \in N\}$, where $N = \{1, 2, \dots, n\}$ is the set of all jobs. This structure defines a batch-based reentrant hybrid flow shop scheduling problem, as illustrated in Fig. 1.

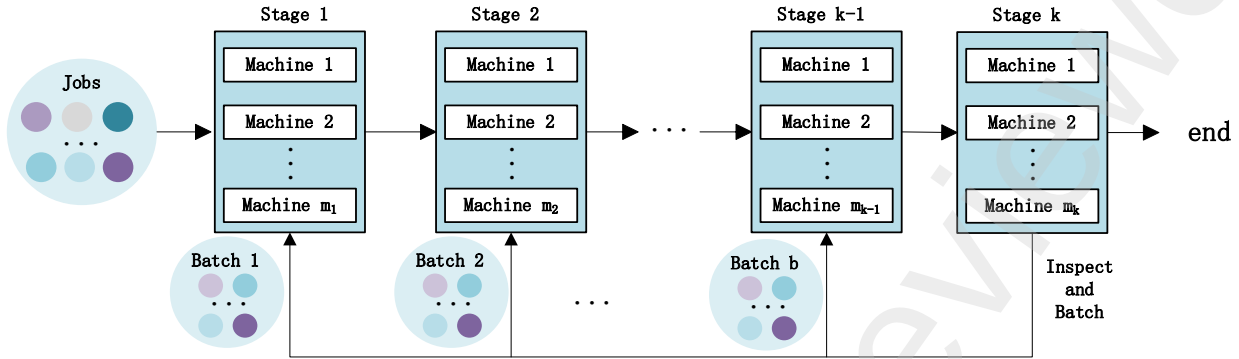


Fig. 1 Schematic diagram of SMO-RHFSP-BDST.

Two types of setup operations are considered:

- Initial-to-rework setup: After completing all initial processing tasks, each machine must immediately perform a setup operation with duration τ_0 , and then remain idle until rework jobs are released. This ensures resource continuity and avoids machine conflicts.
- Inter-batch setup: When a machine switches from processing rework jobs of batch B_p to batch B_q , a batch-dependent setup time τ_{pq} is required.

As machine operations account for most of the energy consumption in manufacturing systems, the energy usage of machines during the scheduling horizon is explicitly modeled. Each machine can be in one of four mutually exclusive states, each with distinct power characteristics:

- Processing state: The base processing power at stage j is denoted by p_e^j . Following the power conversion factor proposed by Mansouri et al. (2016), the actual processing power of machine m at stage j is $p_{jm}^{proc} = p_e^j \cdot \lambda_{jm}$. Typically, machines with higher processing speeds (v_{jm}) exhibit larger power coefficients (λ_{jm}).
- Idle state: When a machine is idle but powered on, it consumes a idle power p_{jm}^{idle} .
- Setup state: During setup operations, the machine consumes power p_{jm}^{setup} .
- Shutdown-restart state: If the idle time of machine jm exceeds a shutdown threshold TB_{jm} , the machine is turned off to save energy. Restarting the machine incurs additional energy consumption with restart power $p_{jm}^{restart}$.

The model accounts for a TOU electricity pricing scheme. Electricity prices follow a Critical Peak Pricing (CPP) policy, where the electricity price in time period $t \in T$ is denoted by f_t . The TEC is defined as $TEC = \sum_{j \in J} \sum_{m \in M} \sum_{t \in T} E_{jm}^t \times f_t$, where E_{jm}^t represents the energy consumption of machine jm during period t . Carbon-related cost is evaluated under an allowance-based mechanism. The plant is assigned a carbon emissions allowance EA , and any emissions exceeding this quota are charged at the prevailing carbon market price C_p . The CTC is expressed as $CTC = \left(\mu \times \sum_{j \in J} \sum_{m \in M} \sum_{t \in T} E_{jm}^t - EA \right) \times C_p$. Where μ denotes the carbon emission coefficient that converts electricity consumption into equivalent carbon emissions.

The following assumptions are made:

- All jobs and machines are available at the initial time;
- Job processing is non-preemptive;
- A job may enter the next stage only after completing the previous one;
- At each stage, each job can be assigned to only one machine;
- Each machine can handle at most one job at any moment;
- Machines operate in four mutually exclusive states: processing, idle, setup, and restart;
- Higher machine processing speeds lead to higher energy consumption;
- The processing time of rework jobs is α times that of initial processing, while power characteristics remain unchanged;
- An infinite-capacity buffer exists after the completion of the initial processing phase;
- All rework batching decisions are completed after initial processing is finished.

3.2. Mathematical Model of the SMO-RHFSP-BDST

The model uses the following notation and terms.

Notation	Description
Sets	
N	Set of jobs, $i \in N = \{1, 2, \dots, n\}$
N_r	Subset of jobs requiring rework, $N_r \subseteq N$
J	Set of processing stages, $j \in J = \{1, 2, \dots, k\}$
R	Set of rework starting stages, $r_i \in R = \{1, 2, \dots, k-1\}$
M_j	Set of machines at stage, $m \in M_j = \{1, 2, \dots, m_j\}$
M	Set of all machines, $jm \in M = \{m_1, \dots, m_i\}$
T	Set of discrete time periods, $t \in T$
B_s	Set of jobs in rework batch, $\beta_i \in B_s = \{i r_i = s, i \in N\}$, where $s \in \{1, 2, \dots, b\}$ denotes the batch index and b is the total number of rework batches.
Parameters	

P_{jm}^0	Initial processing time of job i at stage j on machine m
v_{jm}	Processing speed coefficient of machine jm
α	Rework time multiplier, $\alpha \geq 1$
τ_0	Setup time between initial processing and rework
τ_{pq}	Setup time for switching from rework batch p to batch q
λ_{jm}	Power coefficient of machine jm
P_e^j	Base processing power at stage j
P_{jm}^{proc}	Processing power of machine jm
P_{jm}^{idle}	Idle power consumption of machine jm
P_{jm}^{setup}	Power consumption during setup jm
$P_{jm}^{restart}$	Power consumption during machine restart jm
TB_{jm}	Shutdown threshold of machine jm
f_t	Electricity price at time period t
μ	Carbon emission coefficient (0.2 (ton/MWh))
C_p	Carbon trading price (30 (CNY/ton))
EA	Carbon emission allowance (1 ton/(job·stage))
M^0	A sufficiently large positive constant
Decision Variables	
S_{ij}	Start time of job i at stage j
C_{ij}	Completion time of job i at stage j
C_{ik}^0	Completion time of job i at stage k in initial processing
T_{init}	Completion time of the initial processing phase
T_{jm}^I	Completion time of the last initial job on machine jm
T_{jm}^E	Completion time of initial-to-rework setup on machine jm
Δt	Idle time duration of machine jm
x_{ijm}	1 if job i is assigned to machine m at stage j ; 0 otherwise
δ_{ij}	1 if job i is in rework state at stage j ; 0 otherwise
y_t^{jm}	1 if machine jm is processing at time t ; 0 otherwise
z_t^{jm}	1 if machine jm is idle at time t ; 0 otherwise
w_t^{jm}	1 if machine jm is in setup state at time t ; 0 otherwise
u_t^{jm}	1 if machine jm is in restart state at time t ; 0 otherwise
$o_{ii'}^{jm}$	1 if job i precedes job i' on machine jm ; 0 otherwise
$\gamma_{ii'}^{jm}$	1 if job i precedes job i' on machine jm belong to different rework batches; 0 otherwise
C_{max}	Makespan

Objective:

$$\text{Min}(C_{max}, TEC, CTC) \quad (1)$$

Subject to:

$$C_{ij} - S_{ij} = \sum_{m \in M_j} x_{ijm} \cdot \frac{P_{jm}^0}{v_{jm}}, \quad \forall i \in N, j \in J, \delta_{ij} = 0 \quad (2)$$

$$C_{ij} - S_{ij} = \alpha \cdot \sum_{m \in M_j} x_{ijm} \cdot \frac{P_{jm}^0}{v_{jm}}, \quad \forall i \in N_r, j \geq r_i, \delta_{ij} = 1 \quad (3)$$

$$S_{i,j+1} \geq C_{ij}, \quad \forall i \in N, j \in \{1, \dots, k-1\} \quad (4)$$

$$\sum_{m \in M_j} x_{ijm} = 1, \quad \forall i \in N, j \in J \quad (5)$$

$$S_{i,j} \geq C_{ij} + M^0 (3 - x_{ijm} - x_{i,jm} - o_{ii,jm}), \quad \forall i, i' \in N, i \neq i', j \in J, m \in M_j \quad (6)$$

$$S_{ij} \geq C_{ij} + M^0 \left(2 - x_{jm} - x_{i,jm} + o_{i,jm} \right), \quad \forall i, i' \in N, i \neq i', j \in J, m \in M_j \quad (7)$$

$$\delta_{ij} = \begin{cases} 0, & \forall i \in N, j \in J \\ 1, & \forall i \in N_r, j \geq r_i \end{cases} \quad (8)$$

$$S_{i_r} \geq T_{init}, \quad \forall i \in N_r \quad (9)$$

$$T_{init} = \max_{i \in N} C_{ik}^0 \quad (10)$$

$$T_{jm}^I = \max_{i \in N, \delta_{ij}=0} \{C_{ij} \cdot x_{jm}\} \quad (11)$$

$$T_{jm}^E = T_{jm}^I + \tau_0 \quad (12)$$

$$S_{ij} \geq T_{jm}^E \cdot x_{jm}, \quad \forall i \in N_r, j \geq r_i, m \in M_j \quad (13)$$

$$\gamma_{ii}^{jm} \geq x_{jm} + x_{i,jm} - 1, \quad \forall i, i' \in N_r, \beta_i \neq \beta_{i'}, j \in J, m \in M_j \quad (14)$$

$$S_{i'j} \geq C_{ij} + \tau_{\beta_i \beta_{i'}} \cdot \gamma_{ii}^{jm} \cdot o_{ii}^{jm}, \quad \forall i, i' \in N_r, \beta_i \neq \beta_{i'}, j \in J, m \in M_j \quad (15)$$

$$y_t^{jm} + z_t^{jm} + w_t^{jm} + u_t^{jm} \leq 1, \quad \forall j, m, t \quad (16)$$

$$y_t^{jm} \geq x_{jm}, \quad \forall i \in N, j \in J, m \in M_j, S_{ij} \leq t \leq C_{ij} \quad (17)$$

$$w_t^{jm} = 1, \quad \forall i \in N, j \in J, m \in M_j, t \in \left[T_{jm}^I, T_{jm}^E \right] \vee \left[C_{ij}, S_{i,j} \right] \quad (18)$$

$$z_t^{jm} = 0, \quad \forall t \in \Delta t \quad (19)$$

$$u_t^{jm} = 1, \quad \Delta t > TB_{jm}, t = \text{restart time} \quad (20)$$

$$x_{jm}, \delta_{ij}, y_t^{jm}, z_t^{jm}, w_t^{jm}, u_t^{jm}, o_{ii}^{jm}, \gamma_{ii}^{jm} \in \{0, 1\}, \quad \forall i, j, m, t \quad (21)$$

$$C_{ij}, S_{ij} \geq 0, \quad \forall i, j \quad (22)$$

$$C_{\max} \geq C_{ij}, \quad \forall i \in N, j = k \quad (23)$$

$$TEC = \sum_{j \in J} \sum_{m \in M} \sum_{t \in T} \left(p_{jm}^{proc} \cdot y_t^{jm} + p_{jm}^{idle} \cdot z_t^{jm} + p_{jm}^{setup} \cdot w_t^{jm} + p_{jm}^{restart} \cdot u_t^{jm} \right) \cdot f_t \quad (24)$$

$$CTC = \left(\mu \times \sum_{j \in J} \sum_{m \in M} \sum_{t \in T} \left(p_{jm}^{proc} \cdot y_t^{jm} + p_{jm}^{idle} \cdot z_t^{jm} + p_{jm}^{setup} \cdot w_t^{jm} + p_{jm}^{restart} \cdot u_t^{jm} \right) - EA \right) \times C_p \quad (25)$$

Constraints (2) and (3) define the processing time calculations for jobs in the initial processing stage and the rework stage, respectively, where the processing time of rework jobs is multiplied by a factor α . Constraint (4) ensures that each job follows the predefined technological order among processing stages. Constraint (5) guarantees that each job is assigned to exactly one machine at each stage. Constraints (6) and (7) determine the processing sequence of jobs assigned to the same machine, thereby preventing processing overlap. Constraint (8) identifies whether a job is in the rework state at a given stage. Constraints (9) and (10) ensure that all rework operations can only start after the completion of the initial processing phase and define the completion time of the initial processing stage. Constraints (11) and (12) incorporate the initial-to-rework setup operation required after the initial processing phase and determine the corresponding setup completion time. Constraint (13) ensures that the starting time of rework jobs is no earlier than the completion time of the initial-to-rework setup on the assigned machine. Constraints (14) and (15) model the batch-dependent setup times required when rework jobs from different batches are consecutively processed on the same machine. Constraints (16) and (20) describe the operating states of machines, ensuring that each machine can only remain in one of four mutually exclusive states at any moment. These constraints also capture machine shutdown and restart behaviors when idle times exceed predefined thresholds. Constraints (21) and (22) define the domains of the decision variables. Constraint (23) specifies the makespan. Constraints (24) and (25) calculate the TEC and the CTC, respectively, by considering machine power

consumption under different operating states, time-varying electricity prices, and carbon emission allowance constraints.

4. Solution Method for the SMO-RHFSP-BDST Based on MQNSGA-II

The MQNSGA-II algorithm consists of several key components, including chromosome encoding and decoding, population initialization, genetic operators, local search, crowding distance evaluation, and a reinitialization mechanism. A two-phase chromosome representation is adopted to simultaneously encode the scheduling sequences of initial processing and rework operations. The decoding procedure maps chromosomes into feasible schedules. A hybrid heuristic strategy is employed to generate the initial population, balancing solution quality and population diversity. Problem-specific crossover and mutation operators are designed to enhance search efficiency. During the evolutionary process, a Meta Q-learning-driven local search mechanism is embedded, enabling adaptive selection among multiple neighborhood structures to balance global exploration and local exploitation. To further improve the distribution quality of the obtained Pareto solutions, a hybrid crowding distance evaluation method is proposed by integrating dual-sphere crowding distance (DSCD) in the objective space and maximum extension distance (MED) in the decision space. In addition, a reinitialization mechanism is triggered when search stagnation is detected, effectively preventing premature convergence. The framework of the proposed MQNSGA-II is illustrated in Fig. 2.

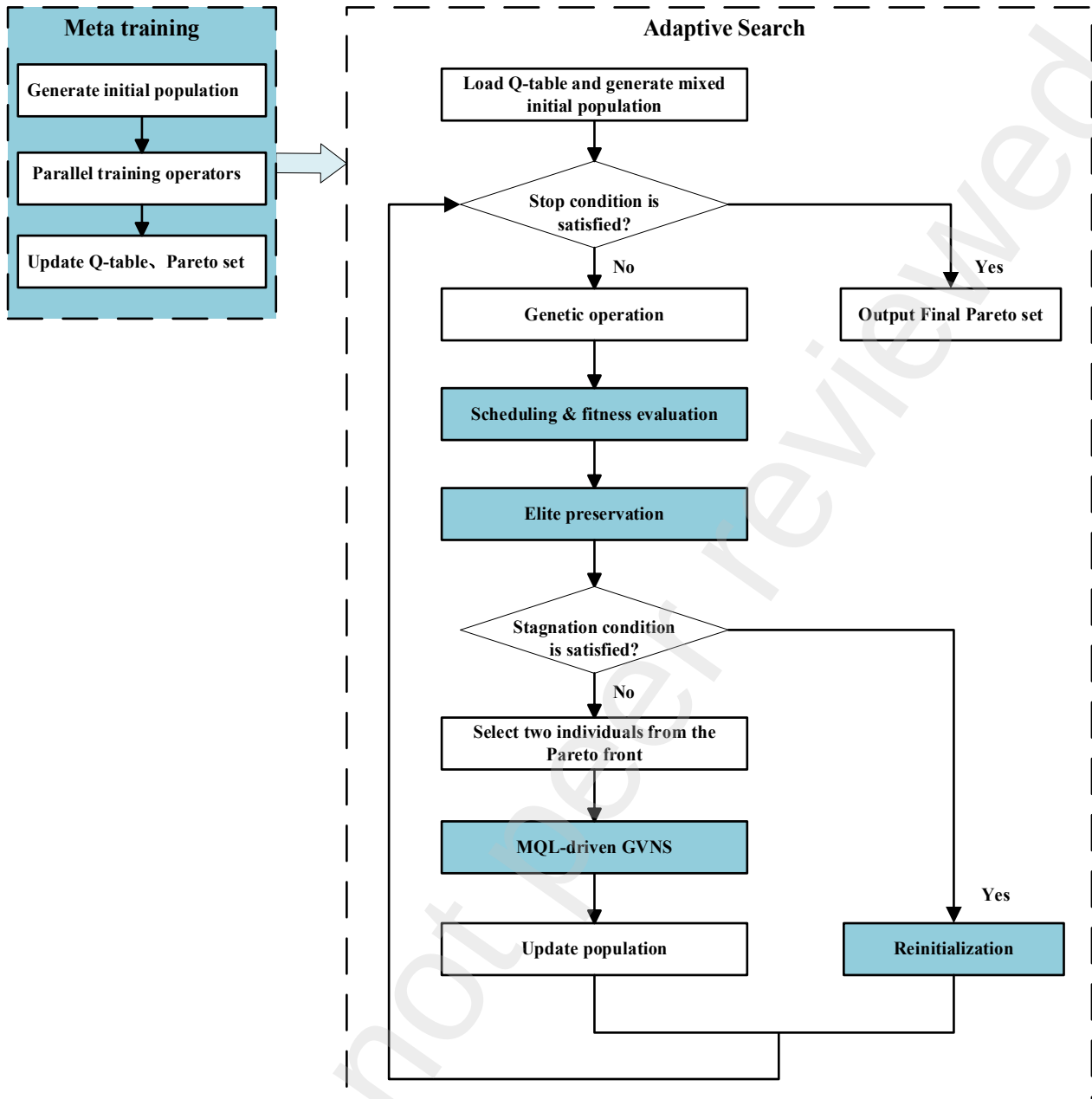


Fig. 2 Framework of MQNSGA-II.

4.1. Encoding and Decoding

For the SMO-RHFSP-BDST, an extended two-phase chromosome representation is designed to simultaneously describe the scheduling orders of initial processing jobs and rework jobs. Correspondingly, a two-phase decoding strategy is proposed to separately handle the initial processing stage and the rework stage, thereby ensuring schedule feasibility while effectively reducing decoding complexity.

4.1.1. Encoding

A job-sequence based permutation encoding is adopted. Each chromosome is divided into two segments: an initial processing job sequence and a rework job sequence. For a scheduling instance with n jobs, among which n_r jobs

require rework ($n_r < n$), the chromosome π has a total length of $n_r + n$, and is represented as $\pi = [\pi^N, \pi^R] = [i_1, \dots, i_n, i_{r_1}, \dots, i_{r_n}]$, where π^N denotes the sequence of jobs for initial processing, and π^R represents the sequence of rework jobs. This two-phase encoding structure allows the initial processing sequence and the rework sequence to be optimized independently. Moreover, it facilitates the design of specialized genetic operators and local search strategies, and supports flexible rework batch scheduling policies.

Algorithm 1: Two-Phase Decoding

Input: The sequence of chromosome $\pi = [\pi^N, \pi^R]$

Output: Feasible schedule ψ

% Phase 1: Initial Processing Decoding

1. Initialize job completion time and machine available time $C_i, A_{jm} \leftarrow 0$

2. **For** i in π^N **do**

3. **For** $j = 1$ to k **do**

4. $m^* \leftarrow \arg \min_{m \in M_j} \{ \max(C_i, A_{jm}) + p_{jm} \}$

5. $S_{ij} \leftarrow \max(C_i, A_{jm^*})$

6. $C_{ij} \leftarrow S_{ij} + p_{jm^*}$

7. $C_i \leftarrow C_{ij}, A_{jm^*} \leftarrow C_{ij}$

8. record operation in ψ

9. **End for**

10. **End for**

11. $T_{init} \leftarrow \max C_{ik}$

%Phase 2: Rework Decoding

12. Initialize $A_{jm} \leftarrow \max(T_{jm}^I + \tau_0, T_{init}), C_i^R \leftarrow T_{init}$

13. Current Machine Processing Batch $\beta_{jm} \leftarrow \emptyset$

14. **For** i in π^R **do**

15. **For each** $j \in \text{Rework Path}(i)$ **do**

16. $m^* \leftarrow \arg \min_{m \in M_j} \{ \max(C_i^R, A_{jm}) + \alpha \cdot p_{jm} \}$

17. $S_{ij} \leftarrow \max(C_i^R, A_{jm^*}) + \tau_{\beta_{jm^*}, b_i}$ % b_i is the batch of i

18. $C_{ij} \leftarrow S_{ij} + \alpha \cdot p_{jm^*}$

19. $C_i^R \leftarrow C_{ij}, A_{jm^*} \leftarrow C_{ij}, \beta_{jm^*} \leftarrow b_i$

20. Record operation in ψ

21. **End for**

22. **End for**

23. Return ψ

4.1.2. Decoding

Due to the presence of reentrant processing and batch-dependent setup operations in the SMO-RHFSP-BDST, the decoding process must simultaneously address the temporal linkage between initial processing and rework stages, as well as the additional setup times incurred by batch switching during rework. To this end, a two-phase decoding strategy is proposed, decomposing the decoding process into two relatively independent sub-procedures: initial processing decoding and rework decoding.

- 1) In the initial processing phase, all jobs are processed sequentially through the k stages according to the order specified in π^N . A List Scheduling (LS) heuristic is employed for decoding (Luo et al., 2013), which is widely used in HFSPs. The core scheduling rule of LS is the Earliest Completion Time (ECT) criterion. For the current job i to be scheduled, its earliest completion time on each available machine m at stage j is calculated. The machine that yields the minimum ECT is selected for processing. If multiple machines achieve the same minimum ECT, one of them is randomly selected to enhance solution diversity. After all jobs complete initial processing, the corresponding completion time T_{init} is recorded and serves as the temporal reference for rework scheduling.
- 2) After completing the decoding of the initial processing stage, rework scheduling is performed according to the rework job sequence π^R . A sequential scheduling mechanism is adopted, in which rework jobs are processed one by one following the chromosome defined order and scheduled stage by stage along their respective rework paths. At each rework stage, the machine that yields the minimum job completion time is selected. If a batch change occurs between consecutive rework jobs on a machine, the corresponding batch-dependent setup time is inserted before processing. The complete two-phase decoding procedure is summarized in Algorithm 1.

4.2. Hierarchical Hybrid Population Initialization

The Hierarchical Hybrid Initialization (HHI) strategy integrates three complementary mechanisms that consist of NEH heuristics (Naderi et al., 2010), random generation, and Meta-learning-based knowledge transfer. This design helps maintain both high solution quality and sufficient population diversity. The detailed procedure is presented in Algorithm 2. In this study, the NEH heuristic is extended to the multi-objective setting. Specifically, elite individuals are generated by independently optimizing each objective. To adequately cover different regions of the Pareto front, the initial population P^0 is constructed as follows:

- Multi-objective elite generation: The NEH heuristic is applied separately to each objective to generate elite individuals corresponding to extreme points of the Pareto front:

$$\pi_1^{C_{max}} \leftarrow \text{NEH (makespan)}, \pi_2^{TEC} \leftarrow \text{NEH (TEC)}, \pi_3^{CTC} \leftarrow \text{NEH (CTC)}$$

- Random diversity supplementation: The remaining $N_{pop} - 3$ individuals are generated randomly to enhance population diversity and global exploration capability:

$$P_{base} = \{\pi_1^{C_{max}}, \pi_2^{TEC}, \pi_3^{CTC}\} \cup \{\text{random}\}_{N_{pop}-3}$$

- Meta-learning knowledge transfer: During the adaptive search phase, high-quality historical solutions stored in the Meta-training archive A_{meta} are transferred to the current population. The number of transferred solutions is determined adaptively as

$$N_{meta} = \min\left(|A_{meta}|, \left\lceil \frac{N_{pop}}{2} \right\rceil\right)$$

The final initial population is constructed as:

$$P^0 = A_{meta} [1 : N_{meta}] \cup P_{base} [1 : (N_{pop} - N_{meta})]$$

Algorithm 2: Hierarchical hybrid population initialization

Input: Population size N_{pop} , Meta-archive A_{meta}

Output: Initial population P^0

```

1.  $P_{base} \leftarrow \emptyset$ 
2. For obj from 1 to 3 do
3.    $\pi \leftarrow NEH(obj)$ 
4.    $P_{base} \leftarrow P_{base} \cup \{\pi\}$ 
5. End for
6. For  $i$  from 4 to  $N_{pop}$  do
7.    $\pi \leftarrow random(i)$ 
8.    $P_{base} \leftarrow P_{base} \cup \{\pi\}$ 
9. End for
10. If  $A_{meta}$  exists then
11.    $N_{meta} \leftarrow \min\left(|A_{meta}|, \left\lceil \frac{N_{pop}}{2} \right\rceil\right)$ 
12.    $P^0 \leftarrow A_{meta} [1 : N_{meta}] \cup P_{base} [1 : (N_{pop} - N_{meta})]$ 
13. Else
14.    $P^0 \leftarrow P_{base}$ 
15. End if
16. Return  $P^0$ 

```

The proposed HHI strategy provides three high-quality elite individuals oriented toward different objectives, which mitigates the slow convergence caused by purely random initialization while avoiding premature convergence due to insufficient diversity in single-heuristic initialization. Moreover, by incorporating historical knowledge obtained via Meta-learning, the algorithm starts from a more promising region of the search space, thereby accelerating convergence and improving the quality of the final Pareto front.

4.3. Genetic Operators

To effectively explore the solution space and escape local optima, a position-based crossover (PBX) operator and two mutation operators (swap and scramble) are adopted (Cui et al.,2023; Lu et al.,2022). Furthermore, an adaptive segment-based genetic operation strategy is proposed to balance exploration and exploitation while preserving encoding

feasibility. The PBX operator is well suited for permutation-based encodings and inherits favorable characteristics of parent solutions by preserving partial positional information. Considering the two-phase chromosome structure of the SMO-RHFSP-BDST, the classical PBX operator is extended into a segment-based crossover strategy. Specifically, the standard PBX operation is applied to the segments of initially processed workpieces, while a probabilistic exchange mechanism is adopted for the segments of reworked workpieces. The complete crossover process is illustrated in Fig. 3. To balance global exploration and local refinement, swap mutation and scramble mutation are jointly employed. An adaptive segment-based mutation strategy is designed: with an 80% probability, mutation is applied to the initial processing segment, while with a 20% probability, scramble mutation is performed on the rework segment. The mutation procedures are illustrated in Fig. 4.

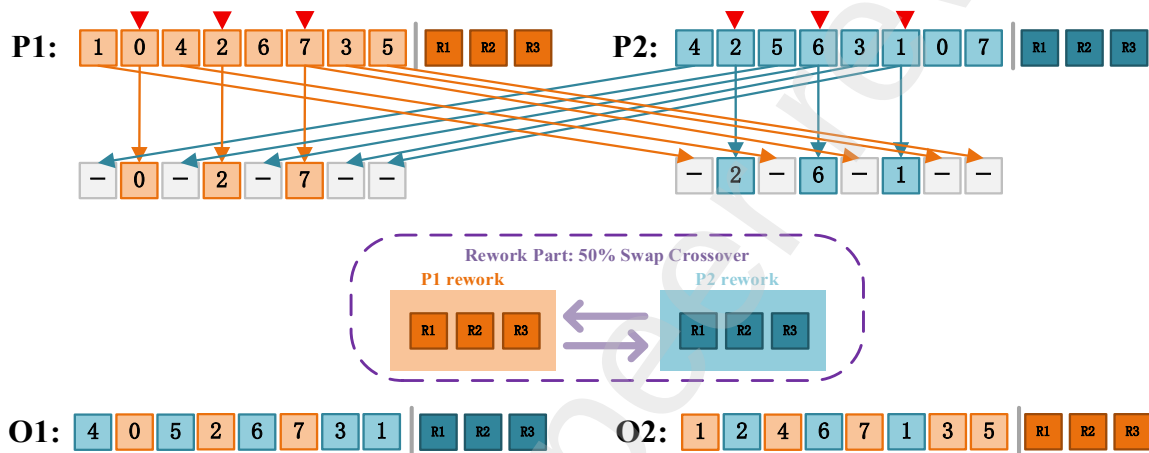


Fig. 3 Illustration of PBX with segmented chromosome structure.

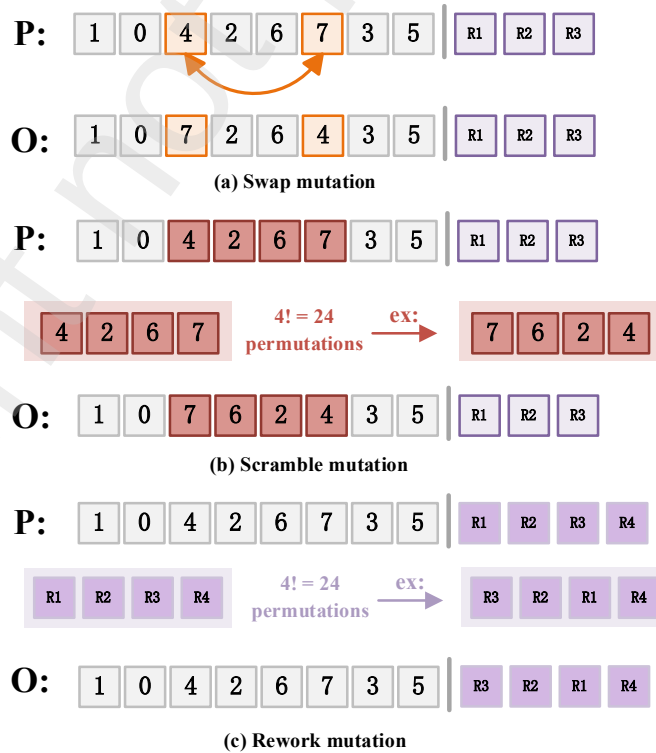


Fig. 4 Illustration of mutation operators.

4.4. Hybrid Decision-Objective Space Crowding Distance and Elite Preservation

The conventional crowding distance used in NSGA-II (Deb et al.,2002), which is solely based on objective space neighborhoods, performs well for bi-objective problems. However, as the number of objectives increases or the Pareto front becomes highly curved, the conventional crowding distance often becomes insufficient for properly characterizing the spatial distribution of solutions. To tackle this problem, a new metric, termed the Hybrid Decision-Objective Space Crowding Distance (HDCD), is proposed. HDCD integrates diversity information from both the objective space and the decision space. For objective space diversity, an improved version of the Dual-Sphere Crowding Distance (DSCD) is adopted and further combined with the Chebyshev distance to strengthen the protection of extreme solutions (Lei et al.,2014). Consider an individual A in the objective space, its two nearest neighbors P_1 and P_2 , and the third nearest neighbor P_3 . The inner sphere radius and outer sphere radius relationship is defined as

$$DSCD[i] = \sigma \times \left\| \frac{P_1 - P_2}{2} \right\|_{\infty}^3 + (1 - \sigma) \times \left[\left(\left\| \frac{A - P_1}{2} \right\|_{\infty}^3 + \left\| \frac{A - P_2}{2} \right\|_{\infty}^3 + \left\| \frac{A - P_3}{2} \right\|_{\infty}^3 \right) - \left\| \frac{P_1 - P_2}{2} \right\|_{\infty}^3 \right] \quad (26)$$

where σ is a weighting parameter set to 0.75, and $\|\cdot\|_{\infty}$ denotes the Chebyshev distance. The geometric interpretation of DSCD is illustrated in Fig. 5, where both the curvature of the Pareto front and the local distribution characteristics of solutions are jointly captured by considering the geometric relationships between an individual and its nearest neighbors. However, relying solely on objective space information may lead to insufficient diversity in the decision variable space. To address this issue, the Maximum Extension Distance (MED) is further incorporated (Yue et al.,2025). MED measures the maximum extent to which an individual is spread in the decision space relative to the current population. It reflects the exploration value in the search space. During environmental selection, MED favors individuals that are more widely distributed in the decision space. This reduces the risk of premature convergence in localized regions and enhancing the global search capability of the algorithm.

The proposed HDCD jointly considers objective space distribution quality and decision space diversity. When the number of non-dominated solutions exceeds the population size, individuals with larger HDCD values are preferentially retained to achieve a balance between uniform distribution along the Pareto front and sufficient expansion in the decision space. To ensure comparability, both DSCD and MED are normalized. Let $DSCD_{avg}$ denote the average normalized DSCD value (excluding infinite boundary values), and MED_{avg} denote the average normalized MED value. The HDCD of individual i is defined as

$$HDCD[i] = \begin{cases} \max(DSCD[i], MED[i]), & DSCD[i] > DSCD_{avg} \vee MED[i] > MED_{avg} \\ \min(DSCD[i], MED[i]), & else \end{cases} \quad (27)$$

The core idea of HDCD is illustrated in Fig. 6. If an individual exhibits high diversity in either space, that is, crowding value exceeds the corresponding threshold, it is given a higher survival priority. Conversely, if diversity indicators in both spaces are low, the smaller value is selected as the final score. This avoids preserving individuals located in crowded regions of both spaces. During elite preservation, if the merged population exceeds the predefined population size, individuals in the last truncated non-dominated front are sorted in descending order of HDCD, and the top-ranked individuals are selected to form the next generation.

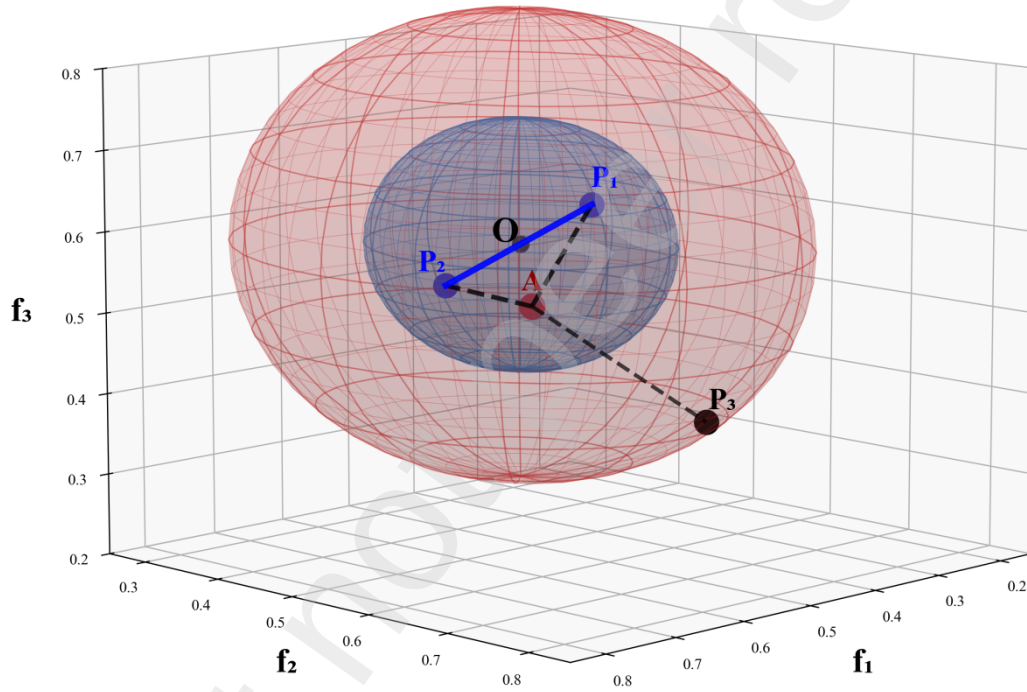


Fig. 5 Geometric interpretation of DSCD.

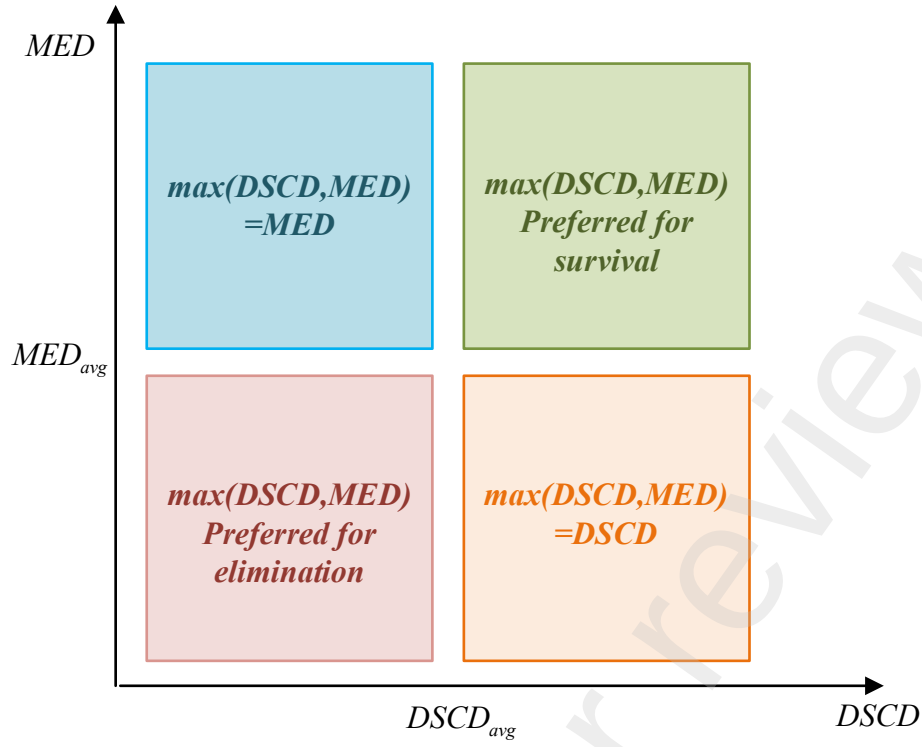


Fig. 6 Schematic diagram of HDCD.

4.5. Local Search Operators

To improve the local exploitation capability of the algorithm in the complex scheduling search space, a set of problem-specific local search operators is designed for the SMO-RHFSP-BDST. As shown in Fig. 7, these operators include one Destruction-Construction (DC) operator and nine VNS operators. The DC operator introduces large scale perturbations by partially destroying and reconstructing the current solution. Specifically, the rework job sequence is kept unchanged, while several jobs are removed from the initial processing sequence and reinserted according to heuristic rules. This helps the algorithm escape local optima. The perturbation intensity is controlled by the number of removed jobs, allowing flexible switching between fine-grained adjustment and global exploration. The VNS operators are categorized into two groups according to their optimization objectives. The first group $N_1 - N_5$ is based on critical path analysis and permutation operations, acting exclusively on the initial processing job sequence to directly improve scheduling performance metrics such as the makespan. The second group $N_6 - N_9$ is designed for energy cost, carbon emission cost, and rework scheduling characteristics. These operators guide the search toward low-energy and low-carbon solutions by adjusting job sequences and rework batch structures. They also improve search diversity in the rework stage. The nine neighborhood operators are defined as follows.

- N_1 (Critical Swap): Randomly selects one critical job and one non-critical job and swaps their positions.
- N_2 (Critical Insertion): Removes a randomly selected critical job and reinserts it into the position of a non-critical

job.

- N_3 (Critical Reversion): Reverses the subsequence between a randomly selected critical job and a non-critical job.
- N_4 (Three-point Heuristic): Randomly selects three consecutive jobs, enumerates all $3! = 6$ permutations, and replaces the original subsequence with the best one.
- N_5 (Segment Crossover): Randomly partitions the job sequence into three segments, enumerates all $3! = 6$ segment orders, and reconstructs the sequence using the best order.
- N_6 (Energy Critical Swap): Swaps the positions of the jobs with the highest and lowest energy consumption to guide the search toward lower energy solutions.
- N_7 (Carbon Critical Swap): Swaps the positions of the two jobs with the highest carbon emissions to reduce the carbon trading cost.
- N_8 (Rework Batch Optimization): Rearranges the rework job sequence by grouping jobs belonging to the same batch, thereby reducing batch switching setup times.
- N_9 (Rework Swap): Randomly swaps two jobs in the rework sequence to perturb the rework scheduling order and enhance diversity.

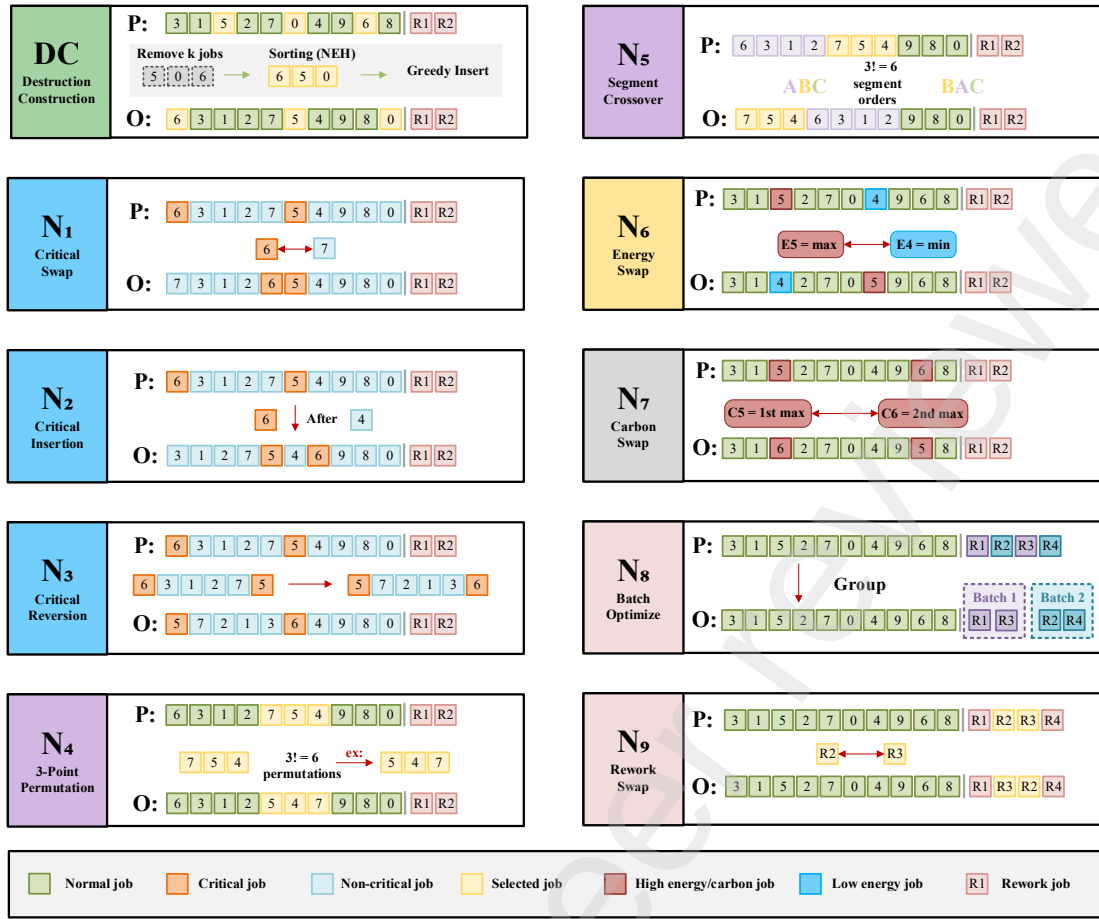


Fig. 7 Illustration of local search operators.

4.6. Meta-Q-Learning Driven GVNS

To fully exploit the synergistic optimization potential of the Destruction-Construction (DC) operator and the nine VNS neighborhood operators designed in Section 4.5, a Meta-Q-Learning-Driven Generalized Variable Neighborhood Search (MQL-GVNS) strategy is proposed. In this approach, the operator selection process is modeled as a Markov Decision Process (MDP) and embedded within a two-stage framework consisting of offline Meta-training and adaptive online search, as illustrated in Fig. 8. During the offline Meta-training stage, the performance of different operators is evaluated in parallel to construct high-quality prior Q-tables. In the subsequent adaptive search stage, the pre-trained knowledge is exploited to guide the alternating execution of DC and VNS operators, enabling dynamic optimization of operator selection strategies (Shao et al., 2024).

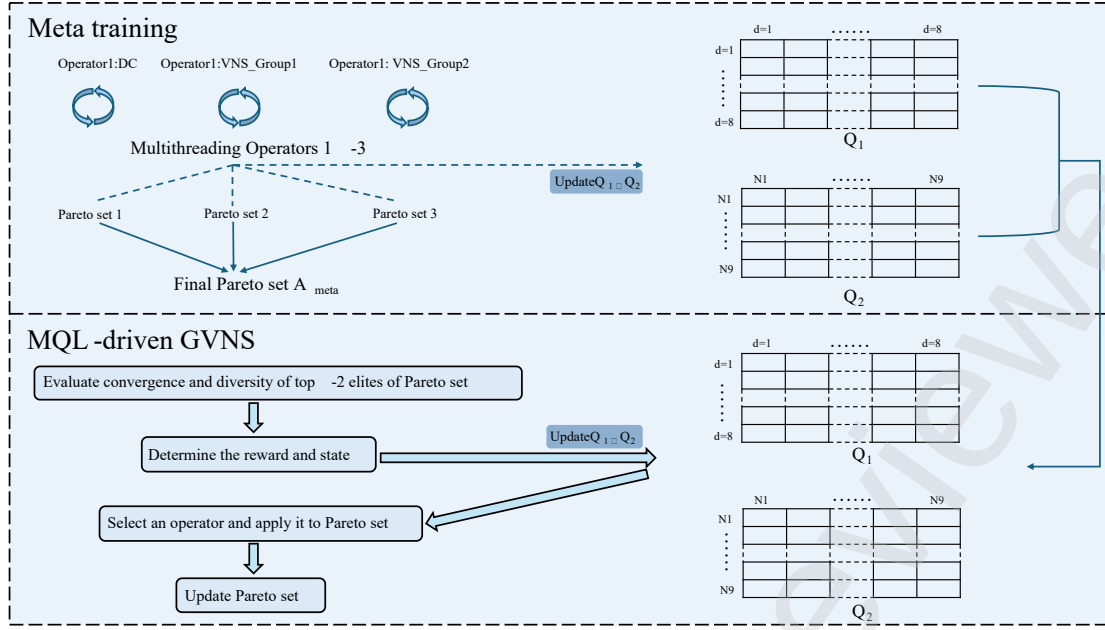


Fig. 8 Framework of MQL-GVNS.

Q-learning estimates state-action value functions to support long-term reward-oriented operator selection (Karimi-Mamaghan et al., 2022). To accommodate the heterogeneous decision variables associated with DC and VNS operators, two state-action spaces and two corresponding Q-tables are constructed: a) DC state-action space $S_1 = A_1 = \{1, 2, 3, 4, 5, 6, 7, 8\}$, where the state and action correspond to the destruction depth d , defined as the number of randomly removed jobs. b) VNS state-action space $S_2 = A_2 = \{N_1, N_2, N_3, N_4, N_5, N_6, N_7, N_8, N_9\}$, where the state and action represent the currently selected neighborhood operator. Accordingly, two Q-tables are maintained as $Q_1 \in R^{|S_1| \times |A_1|}$ and $Q_2 \in R^{|S_2| \times |A_2|}$, where Q_1 guides the selection of destruction depth in the DC operator, and Q_2 controls neighborhood switching in the VNS phase. After executing an action, the standard Q-learning update rule is applied:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \times \left[r_t + \gamma \times \max_a Q(s', a') - Q(s, a) \right] \quad (28)$$

Where α is the learning rate, γ is the discount factor, and r_t denotes the immediate reward. Let A and A' denote the Pareto non-dominated solution sets before and after operator execution, respectively. To simultaneously promote convergence and diversity, a composite reward function is defined as:

$$r_t = \lambda \widehat{\Delta cv}_t + (1 - \lambda) \widehat{\Delta dv}_t, \lambda \in [0, 1] \quad (29)$$

Where $\widehat{\Delta cv}_t \in [0, 1]$ represents the normalized improvement in convergence, and $\widehat{\Delta dv}_t \in [0, 1]$ denotes the normalized improvement in diversity. This reward mechanism encourages the algorithm to approach the true Pareto front while maintaining adequate solution coverage.

Algorithm 3: Meta-training

Input: Initial Pareto set A , learning rate α , discount factor γ , $\theta = \{\theta_1, \theta_2, \theta_3\}$

Output: Q_1, Q_2 , enhanced Pareto set A_{meta}

```
1. Initialize  $Q_1, Q_2, A_i \leftarrow A, R_i \leftarrow \emptyset, \forall i \in \{1, 2, 3\}$ 
3. For  $i = 1, 2, 3$  in parallel do
4.   Repeat
5.      $s, a \leftarrow \text{Select state and action } (\theta_i, A_i)$ 
6.      $A_i' \leftarrow \text{Execute } (a, A_i)$ 
7.      $r, s' \leftarrow \text{Evaluate } (A_i, A_i')$ 
8.      $R_i \leftarrow R_i \cup \{(s, a, r, s')\}$ 
9.      $A_i \leftarrow A_i'$ 
10.  Until termination criterion met
11. End for
12.  $A_{meta} \leftarrow \text{Merge } (A_1, A_2, A_3)$ 
13. For  $i = 1, 2, 3$  do
14.   For each  $(s, a, r, s')$  in  $R_i$  do
15.      $Q(s, a) \leftarrow Q(s, a) + \alpha \times [r + \gamma \times \max_{a'} Q(s', a') - Q(s, a)]$ 
16.   End for
17. End for
18. Return  $Q_1, Q_2, A_{meta}$ 
```

To alleviate the high randomness and slow convergence commonly observed in early stage online learning, an offline Meta-training phase is introduced to pre-train Q_1 and Q_2 . Starting from the same initial Pareto set A_0 , operators are partitioned into three groups and executed in parallel:

- θ_1 : DC operator, responsible for large scale structural perturbation and reconstruction;
- $\theta_2 = \{N_1, N_2, N_3, N_4, N_5\}$, operators focusing on C_{max} optimization;
- $\theta_3 = \{N_6, N_7, N_8, N_9\}$, operators targeting TEC, CTC and rework efficiency.

State-action-reward transitions are sampled to construct experience pools, and batch updates are applied to obtain prior Q-tables. Meanwhile, the non-dominated solutions generated by all threads are merged to form an enhanced initial Pareto set A_{meta} , which is used to initialize the adaptive search phase. In the adaptive GVNS phase, the pre-trained Q-tables guide the alternating execution of DC and VNS operators. In each generation, two elite individuals are selected from the current Pareto front and enhanced using MQL-GVNS. For each selected elite solution x , the following alternating optimization is performed:

- DC phase: A destruction depth d is selected via probabilistic sampling based on Q_1 , and the DC operator is applied to generate a candidate solution x' . If x' improves upon the current solution, Q_1 is updated and the new

solution is accepted.

- VNS phase: A neighborhood operator is selected via probabilistic sampling based on Q_2 , and VNS is applied to generate a candidate solution x'' . If x'' improves upon the current solution, Q_2 is updated and the new solution is accepted; otherwise, the inferior solution is accepted with probability ε to maintain search diversity.

The detailed procedures of the offline Meta-training and adaptive GVNS phases are summarized in Algorithm 3 and Algorithm 4, respectively.

Algorithm 4: Adaptive GVNS

Input: Enhanced Pareto set A_{meta} , Q_1, Q_2 from Meta-training, discount factor γ , exploration rate ε

Output: Final Pareto-optimal set A^*

```

1. Repeat
2.    $\{x_1, x_2\} \leftarrow$  Select top 2 from  $A_{meta}$ 
3.   For  $i = 1$  to 2 do
4.      $a_i \leftarrow$  choose action  $(s_i, Q_i)$ ,  $x'_i \leftarrow DC(x_i, a_i)$ 
5.     If improved  $(x'_i, x_i)$  do
6.        $Q_i(s_i, a_i) \leftarrow Q_i(s_i, a_i) + \alpha_{adaptive} \times [r + \gamma \times \max_a Q_i(s_i, a) - Q_i(s_i, a_i)]$ 
7.     End if
8.      $a_2 \leftarrow$  choose action  $(s_2, Q_2)$ ,  $x'_i \leftarrow VNS(x'_i, a_2)$ 
9.     If improved  $(x'_i, x_i)$  do
10.       $Q_2(s_2, a_2) \leftarrow Q_2(s_2, a_2) + \alpha_{adaptive} \times [r + \gamma \times \max_a Q_2(s_2, a) - Q_2(s_2, a_2)]$ 
11.    End if
12.  End for
13.   $A \leftarrow$  Update with enhanced  $\{x_1, x_2\}$ 
14. Until termination criterion met
15. Return  $A^*$ 

```

4.7. Resurrection-Based Reinitialization Strategy

To prevent premature convergence and maximize the utilization of historical computational resources, a Resurrection-Based Reinitialization (RR) strategy is proposed, inspired by the subpopulation reinitialization mechanism of Cui et al. (2023). This strategy is activated when the population fails to improve the global best solution over multiple consecutive generations.

The quality of the best individual in each generation is evaluated using a weighted objective value C_w . If no improvement is observed for T consecutive generations, population stagnation is detected and reinitialization is triggered. During reinitialization, individuals are first sorted according to C_w . The top 50% elite individuals P_{elite} are retained to preserve high-quality solution information, while the remaining 50% inferior individuals are eliminated. The complete information of eliminated individuals (chromosome encodings and objective values) is stored in a resurrection

pool for subsequent reuse. In the population regeneration phase, $0.5 \times N_{pop}$ new individuals are generated using the NEH heuristic.

To further exploit historical high-quality individuals and reduce redundant computation, a probability driven resurrection mechanism is designed. Eliminated individuals stored in the resurrection pool are sampled without replacement with probability p_{rv} , replacing at most 50% of the newly generated individuals. The resurrection probability is defined as:

$$p_{rv} = \eta \times \max \left(0, \frac{C_{avg} - C_{best}}{C_{ind} - C_{best}} \right) \times \left(\frac{RT_{now}}{RT_{max}} \right)^{\tau} \quad (30)$$

Where η is the base resurrection probability, C_{ind} denotes the weighted objective value of an eliminated individual, C_{avg} and C_{best} represent the average and best weighted objective values of the elite population, respectively. RT_{now} and RT_{max} denote the current runtime and the maximum allowable runtime, and τ is a time-decay factor. This probability design ensures a low resurrection rate during the early exploration stage to prevent poor solutions from re-entering the population, while gradually increasing the resurrection probability in the later exploitation stage to fully leverage historically promising solutions.

5. Numerical Analysis

This section presents a series of comprehensive computational experiments to validate the effectiveness and superiority of the proposed MQNSGA-II algorithm. The experimental setup is first introduced, including test instances, parameter settings, and performance indicators. Next, the key parameters of the algorithm are systematically tuned using the Taguchi experimental design method. Then, four groups of ablation experiments are conducted to analyze the contribution of each innovative component of the algorithm. Finally, the performance of MQNSGA-II is compared with four state-of-the-art and classical multi-objective optimization algorithms (NSGA-II, JAYA, MOEA/D, and QVNS_NSGA-II) (Deb et al.,2002; Li et al.,2023; Pan et al.,2023; Wang et al.,2021), and statistical tests are performed on the experimental results to further verify the stability and significant advantages of the proposed method.

5.1. Experimental Settings

The experiments were carried out on a PC with an Intel Core Ultra CPU (2.00 GHz) and 32 GB of RAM, using Python 3.12. Three different scales of test instances (large, medium, and small) were designed. The parameters of the test instances include the number of jobs $n \in \{20, 50, 80\}$, the number of stages $k \in \{3, 5\}$, and the number of machines

per stage $m \in \{3, 4, 6\}$, totaling 18 instances. For example, the instance 20-3-3 represents 20 jobs processed sequentially through three stages, with three parallel machines at each stage. Each instance was run 15 times independently. The key parameters for each test instance are shown in Table 2, where the processing times and machine power were randomly generated using uniform distributions to increase the diversity and representativeness of the instances (Li et al.,2023).

Table 2 Data settings of instances.

Parameter	Setting
Number of jobs	(20, 50, 80)
Number of stages	(3, 5)
Number of machines at each stage	(3, 4, 6)
Processing time of each operation	U[5,10] (hour)
Power of machine (base)	U[5,10] (10^5 W)
Processing speed	(1.2,1.0,0.8)
Conversion rate	(1.5,1.0,0.6)
Idle power of machine	2
Reset power of machine	4
Setup power of machine	3
Break-even point	2
Setup time for rework batch	5

5.2. Performance indicators

To comprehensively evaluate the quality of the non-dominated solution sets obtained by the proposed algorithm, two performance indicators widely used in multi-objective optimization are employed: Hypervolume (HV) and Inverted Generational Distance Plus (IGD⁺). The HV metric simultaneously measures both the convergence and diversity of a solution set, whereas IGD⁺ overcomes the potential Pareto inconsistency issues that may arise with the traditional IGD by adopting an improved distance definition, making it more suitable for comparative algorithmic analysis. Each instance is independently run 15 times with the same parameter settings, and the non-dominated solution sets obtained are recorded to calculate the corresponding HV and IGD⁺ values (Fu et al.,2020; Ishibuchi et al.,2019). To facilitate cross-instance and cross-algorithm comparisons, the Relative Percentage Increase (RPI) is used to standardize the results:

$$RPI_i = \frac{f_i - f_b}{f_b} \times 100 \quad (31)$$

Where f_i denotes the HV or IGD⁺ value obtained by a given algorithm for a specific test instance, and f_b represents the benchmark value of that indicator among all comparison algorithms for the same instance.

A larger HV value and a smaller IGD⁺ value indicate better performance and closer proximity to the global Pareto-optimal front. The detailed computation procedures of HV and IGD⁺ are described as follows.

1. Hypervolume (HV)

Let $y^* = (y_1^*, y_2^*, y_3^*)$ denote a reference point in the objective space that is dominated by all Pareto-optimal

solutions. Let X be the obtained non-dominated solution set. The hypervolume indicator is defined as the volume of the region dominated by X :

$$HV(X, y^*) = Volume\left(\bigcup_{x \in X} v(x, y^*)\right) \quad (32)$$

Where $v(x, y^*)$ denotes the hyper-rectangle (i.e., dominated region) defined by solution $x \in X$ and the reference point y^* . A larger HV value indicates superior performance of the solution set in terms of both convergence toward the Pareto front and distribution diversity.

2. Inverted Generational Distance Plus (IGD⁺)

Let X denote the non-dominated solution set obtained by an algorithm, and let X^* represent the reference Pareto front. The IGD⁺ metric is defined as the average minimum positive distance from each reference point to the solution set:

$$IGD^+(X, X^*) = \frac{1}{|X^*|} \sum_{v \in X^*} d^+(v, X) \quad (33)$$

Where $|X^*|$ is the cardinality of the reference Pareto front, and $d^+(v, X)$ denotes the minimum positive distance from reference point v to the solution set X , defined as

$$d^+(v, X) = \min_{x \in X} \sqrt{\sum_{i=1}^m (\max(f_i(x) - f_i(v), 0))^2} \quad (34)$$

A smaller IGD⁺ value indicates that the obtained solution set is closer to the reference Pareto front and exhibits better coverage, and therefore reflects superior algorithmic performance.

Table 3 Parameter values.

Factors	Factors levels			
	1	2	3	4
P_{size}	100	110	120	150
λ	0.8	0.85	0.9	0.95
T	8	12	16	20
Q_{iter}	1	2	3	4
CT	0.5	0.8	1.0	1.2

5.3. Parameter setting

The Taguchi design method is used to systematically calibrate the key parameters of the algorithm. The five key parameters are: (1) Population size P_{size} ; (2) Reinforcement learning discount factor λ ; (3) Reinitialization threshold parameter T ; (4) Q-learning iteration limit Q_{iter} ; (5) CPU time budget coefficient CT , which controls the total running

time of the algorithm. The actual CPU time limit is calculated as $CT \times n \times k^2$, where n is the number of jobs and k is the number of stages. Table 3 lists the five parameters considered in the experiments and their corresponding four levels. Table 4 reports the parameter level combinations generated by the orthogonal array design, together with the average HV values obtained from 15 independent runs on the 20-3-3 test instance for each combination. Table 5 summarizes the response values of each parameter at different levels as well as their corresponding Delta values, based on which the relative importance of the parameters is ranked. The main effects analysis results are illustrated in Fig. 9

Fig. 9. As can be observed, the CPU time budget coefficient CT and the Q-learning iteration limit Q_{iter} exert the greatest influence on algorithmic performance, with Delta values of 0.1563 and 0.1400, respectively, significantly exceeding those of the other parameters. The results imply that computational budget allocation and the depth of Q-learning updates are key factors influencing the search effectiveness. Within the considered parameter ranges and test instance scale, the best parameter configuration is $P_{size} = 120, \lambda = 0.8, T = 16, Q_{iter} = 3, CT = 0.8$.

Table 4 Orthogonal array and HV values.

No.	P_{size}	λ	T	Q_{iter}	CT	HV
1	100	0.80	8	1	0.5	0.539469
2	100	0.85	12	2	0.8	0.606553
3	100	0.90	16	3	1.0	0.701431
4	100	0.95	20	4	1.2	0.466759
5	110	0.80	12	3	1.2	0.767545
6	110	0.85	8	4	1.0	0.675460
7	110	0.90	20	1	0.8	0.754805
8	110	0.95	16	2	0.5	0.490537
9	120	0.80	16	4	0.8	0.737623
10	120	0.85	20	3	0.5	0.681795
11	120	0.90	8	2	1.2	0.574044
12	120	0.95	12	1	1.0	0.705987
13	150	0.80	20	2	1.0	0.600033
14	150	0.85	16	1	1.2	0.631363
15	150	0.90	12	4	0.5	0.442003
16	150	0.95	8	3	0.8	0.680221

Table 5 Parameter response value analysis

Level	P_{size}	λ	T	Q_{iter}	CT
1	0.5786	0.6612	0.6173	0.6579	0.5385
2	0.6721	0.6488	0.6305	0.5678	0.6948
3	0.6749	0.6181	0.6402	0.7077	0.6707
4	0.5884	0.5859	0.6258	0.5805	0.6099
Delta	0.0963	0.0753	0.0229	0.1400	0.1563
Rank	3	4	5	2	1

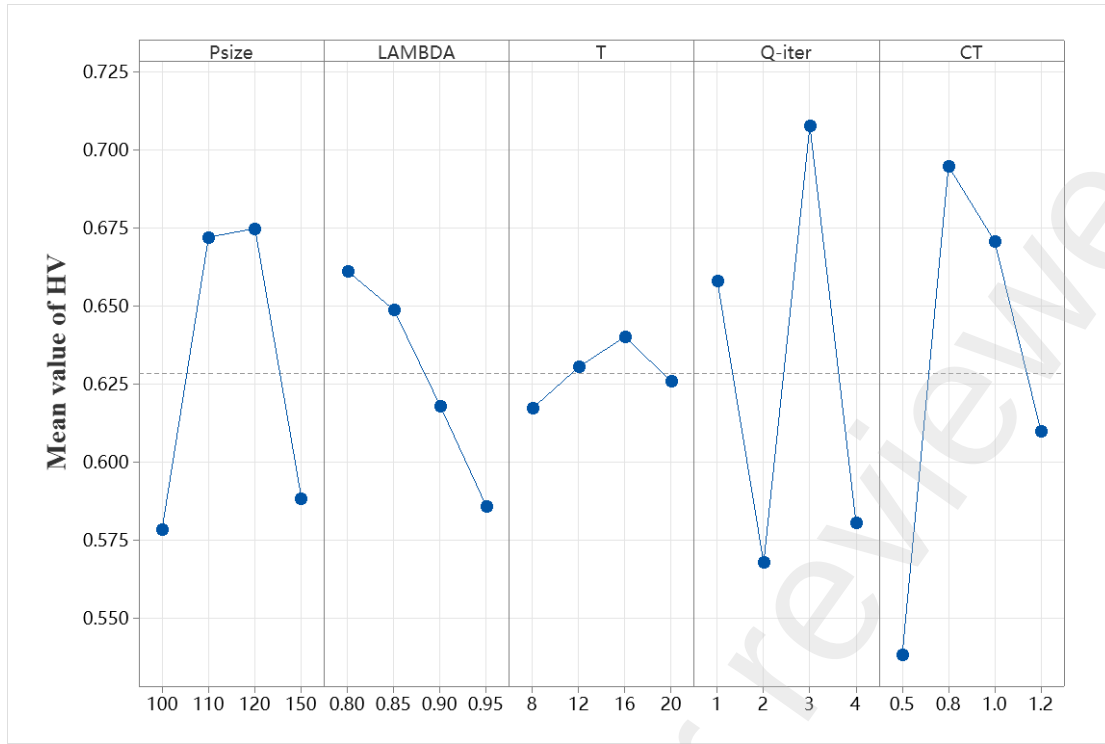


Fig. 9 Trend of the factor level.

5.4. Effectiveness of the proposed components

To quantitatively evaluate the contribution of the key mechanisms embedded in MQNSGA-II to solution quality, four groups of ablation experiments are conducted in this section. Except for the removed or replaced components, all other parameter settings and termination conditions are kept identical. Each algorithm variant is independently executed 15 times on 18 test instances of different scales. The experimental results are summarized in Table 6-Table 9, where the best values are highlighted in bold.

5.4.1. Effectiveness analysis of initialization

To verify the effectiveness of the initialization strategy HHI proposed in Section 4.2, MQNSGA-II is compared with its counterpart using purely random initialization, denoted as MQ_INIT. As reported Table 6, MQNSGA-II achieves the best HV values on all 18 instances and attains the best or near-best IGD⁺ values on 17 instances. These results suggest that HHI effectively improves the quality and stability of the non-dominated solution sets in the early search stage. Further analysis by problem scale reveals that the advantages of HHI become more pronounced as the problem size increases. Compared with MQ_INIT, the average HV improvement is approximately 16.1% for small scale instances (20 jobs), 25.7% for medium scale instances (50 jobs), and further increases to about 29.1% for large scale instances (80 jobs). Correspondingly, IGD⁺ is reduced by about 22.7%, 23.5%, and 25.4%, respectively. This trend indicates that high-quality initialization provides a superior starting Pareto set in more complex search spaces, thereby enhancing the overall optimization efficiency. To exclude the influence of random fluctuations, statistical tests are further conducted on the RPI

values of HV and IGD⁺. The 95% least significant difference (LSD) confidence intervals are plotted. As shown in Fig. 10, the confidence intervals of the two methods do not overlap. This indicates that the observed performance differences are statistically significant and further validating the effectiveness of the HHI strategy.

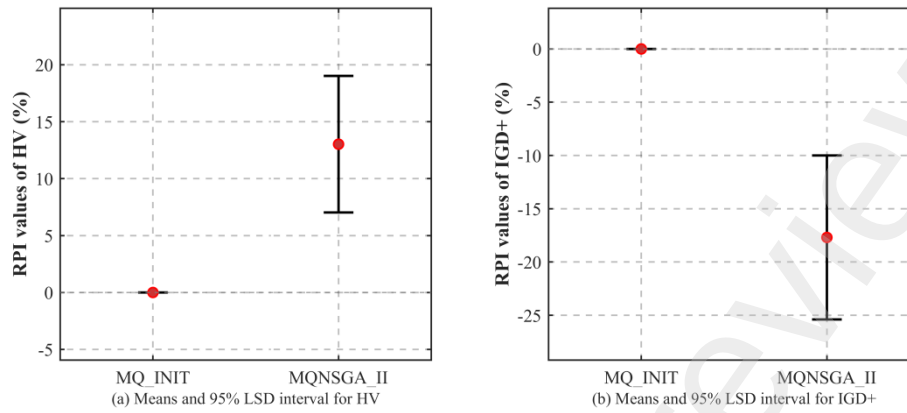


Fig. 10 Confidence intervals for MQNSGA-II and MQ_INIT.

Table 6 Comparison of the MQNSGA-II and MQ_INIT results.

Instance	HV		IGD ⁺	
	MQ_INIT	MQNSGA-II	MQ_INIT	MQNSGA-II
20-3-3	0.540351	0.60884	0.189443	0.155784
20-3-4	0.533077	0.602968	0.176524	0.126093
20-3-6	0.540071	0.615533	0.131459	0.11392
20-5-3	0.468299	0.554066	0.175238	0.128727
20-5-4	0.515653	0.610769	0.169767	0.135287
20-5-6	0.48888	0.588498	0.282889	0.19931
50-3-3	0.517534	0.595887	0.154036	0.129257
50-3-4	0.511709	0.584054	0.14148	0.14168
50-3-6	0.473718	0.588331	0.225581	0.157149
50-5-3	0.381332	0.540683	0.271558	0.174807
50-5-4	0.532785	0.595399	0.16536	0.140334
50-5-6	0.363672	0.535377	0.309934	0.173886
80-3-3	0.435836	0.633037	0.308932	0.164051
80-3-4	0.438517	0.594072	0.259634	0.182633
80-3-6	0.464457	0.538864	0.175383	0.166755
80-5-3	0.509901	0.652894	0.247392	0.183327
80-5-4	0.485403	0.556556	0.149673	0.140007
80-5-6	0.381827	0.516042	0.18681	0.11444
Avg	0.476835	0.583993	0.206727	0.151525

5.4.2. Effectiveness Analysis of Crowding Distance Calculation

To evaluate the effectiveness of the HDCD mechanism proposed in Section 4.4, MQNSGA-II is compared with MQ_CD, which employs the classical crowding distance(Deb et al.,2002). Table 7 shows that MQNSGA-II achieves the best HV values on all instances and outperforms MQ_CD on IGD⁺ in 16 instances. This indicates that HDCD is more effective in maintaining uniformity and coverage of non-dominated solution sets, thereby improving the quality of the Pareto-approximate fronts. From the perspective of problem scale, the advantages of HDCD are more evident for medium and large scale instances. For example, in the 50-job instances, the average HV improvement reaches approximately 24.0% and the IGD⁺ reduction is about 22.8%. Similar improvements are observed for the 80-job instances. These results suggest that when the Pareto set becomes larger and the objective space structure more complex, the classical crowding distance

is no longer enough. It only uses the distance information in the objective space. In contrast, HDCD integrates both objective space and decision space information. This allows it to guide the environmental selection process more effectively. As shown in Fig. 11, Statistical significance tests based on 95% LSD confidence intervals further confirm the significance of these differences. This supports the effectiveness of the proposed HDCD mechanism.

Table 7 Comparison of the MQNSGA-II and MQ_CD results.

Instance	HV		IGD ⁺	
	MQ_CD	MQNSGA-II	MQ_CD	MQNSGA-II
20-3-3	0.55419	0.60884	0.163054	0.155784
20-3-4	0.523329	0.602968	0.165241	0.126093
20-3-6	0.581343	0.615533	0.110774	0.11392
20-5-3	0.495342	0.554066	0.151764	0.128727
20-5-4	0.540507	0.610769	0.161149	0.135287
20-5-6	0.541323	0.588498	0.234779	0.19931
50-3-3	0.543704	0.595887	0.125766	0.129257
50-3-4	0.49578	0.584054	0.152791	0.14168
50-3-6	0.448459	0.588331	0.224243	0.157149
50-5-3	0.406869	0.540683	0.263493	0.174807
50-5-4	0.46619	0.595399	0.215784	0.140334
50-5-6	0.429736	0.535377	0.263602	0.173886
80-3-3	0.503554	0.633037	0.238738	0.164051
80-3-4	0.452395	0.594072	0.259367	0.182633
80-3-6	0.43798	0.538864	0.195604	0.166755
80-5-3	0.549712	0.652894	0.213226	0.183327
80-5-4	0.466179	0.556556	0.171262	0.140007
80-5-6	0.412979	0.516042	0.161728	0.11444
Avg	0.491643	0.583993	0.192909	0.151525

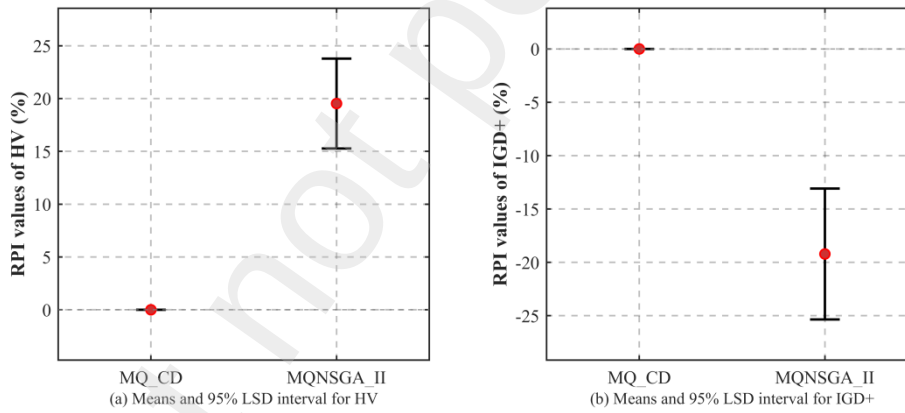


Fig. 11 Confidence intervals for MQNSGA-II and MQ_CD.

5.4.3. Effectiveness Analysis of MQL-GVNS

To assess the contribution of the MQL-GVNS mechanism proposed in Section 4.6, MQNSGA-II is compared with a baseline algorithm employing conventional Q-learning, denoted as QNSGA-II. According to Table 8, MQNSGA-II attains the best HV values on all instances and outperforms the baseline in terms of IGD⁺ on 16 instances. These results indicate that the prior knowledge provided by the Meta-training stage effectively improves neighborhood scheduling efficiency and search stability. Further analysis by scale reveals that the advantage of MQL-GVNS becomes increasingly evident as the problem size increases. Compared with the conventional Q-GVNS, the average HV improvement increases from approximately 10.9% (20-job instances) to 19.2% (80-job instances), while the corresponding IGD⁺ reduction

increases from about 12.6% to 15.5%. Statistical tests based on RPI values and 95% LSD confidence intervals Fig. 12 show no overlap between the two methods, confirming the statistical significance of the observed improvements and further supporting the effectiveness of the proposed MQL-GVNS mechanism.

Table 8 Comparison of the MQNSGA-II and QNSGA-II results.

Instance	HV		IGD ⁺	
	QNSGA-II	MQNSGA-II	QNSGA-II	MQNSGA-II
20-3-3	0.572834	0.60884	0.167156	0.155784
20-3-4	0.534316	0.602968	0.159133	0.126093
20-3-6	0.59281	0.615533	0.101909	0.11392
20-5-3	0.496021	0.554066	0.150442	0.128727
20-5-4	0.538788	0.610769	0.171328	0.135287
20-5-6	0.501286	0.588498	0.262816	0.19931
50-3-3	0.558125	0.595887	0.130494	0.129257
50-3-4	0.519859	0.584054	0.148236	0.14168
50-3-6	0.534115	0.588331	0.167602	0.157149
50-5-3	0.464459	0.540683	0.209172	0.174807
50-5-4	0.505393	0.595399	0.191471	0.140334
50-5-6	0.49836	0.535377	0.203548	0.173886
80-3-3	0.491431	0.633037	0.251902	0.164051
80-3-4	0.492924	0.594072	0.233136	0.182633
80-3-6	0.521261	0.538864	0.138005	0.166755
80-5-3	0.518898	0.652894	0.25055	0.183327
80-5-4	0.492643	0.556556	0.144919	0.140007
80-5-6	0.417364	0.516042	0.157222	0.11444
Avg	0.513938	0.583993	0.179947	0.151525

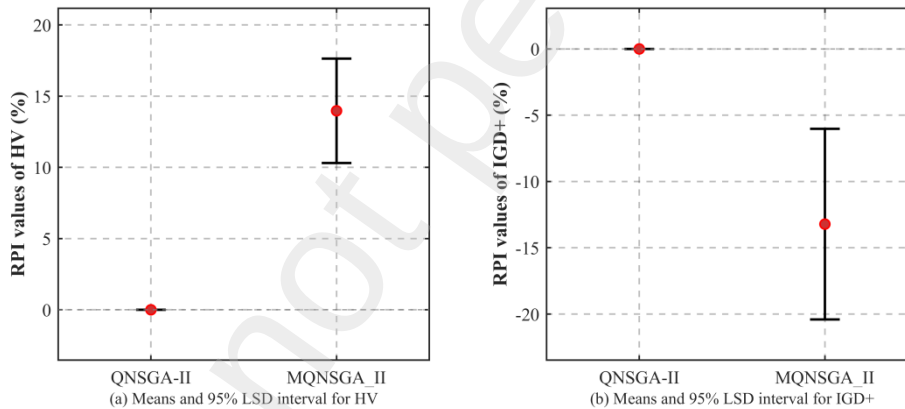


Fig. 12 Confidence intervals for MQNSGA-II and QNSGA-II.

5.4.4. Effectiveness Analysis of Resurrection-Based Reinitialization Strategy

To verify the effectiveness of the RR strategy proposed in Section 4.7, MQNSGA-II is compared with MQ_REINIT, which excludes the RR mechanism. As reported in Table 9, MQNSGA-II exhibits superior overall performance on both HV and IGD⁺. Specifically, it achieves the best HV values on 16 instances and the best or near-best IGD⁺ values on 15 instances, with significantly better average performance than MQ_REINIT. Scale-based analysis shows that RR provides stable performance gains across all instance sizes. For example, in the 20-job instances, the average HV improvement is about 11.3% and IGD⁺ is reduced by approximately 19.9%. Similar improvements are observed in medium and large scale instances. These results demonstrate that the RR mechanism effectively mitigates premature convergence and maintain exploration ability during the search process. This improves both solution quality and robustness. As shown in

Fig. 13, the mean values with 95% LSD confidence intervals reveal statistically significant differences. These results confirm the effectiveness of the RR strategy.

Table 9 Comparison of the MQNSGA-II and MQ_REINIT results.

Instance	HV		IGD ⁺	
	MQ_REINIT	MQNSGA-II	MQ_REINIT	MQNSGA-II
20-3-3	0.578396	0.60884	0.15013	0.155784
20-3-4	0.50878	0.602968	0.193731	0.126093
20-3-6	0.566213	0.615533	0.145571	0.11392
20-5-3	0.497283	0.554066	0.164489	0.128727
20-5-4	0.530571	0.610769	0.180551	0.135287
20-5-6	0.541338	0.588498	0.248629	0.19931
50-3-3	0.623745	0.595887	0.116365	0.129257
50-3-4	0.575729	0.584054	0.148047	0.14168
50-3-6	0.51537	0.588331	0.202618	0.157149
50-5-3	0.38736	0.540683	0.272534	0.174807
50-5-4	0.551438	0.595399	0.155902	0.140334
50-5-6	0.375591	0.535377	0.321126	0.173886
80-3-3	0.634728	0.633037	0.151421	0.164051
80-3-4	0.564644	0.594072	0.21198	0.182633
80-3-6	0.459716	0.538864	0.197391	0.166755
80-5-3	0.588249	0.652894	0.212613	0.183327
80-5-4	0.49087	0.556556	0.180661	0.140007
80-5-6	0.433747	0.516042	0.174233	0.11444
Avg	0.523543	0.583993	0.190444	0.151525

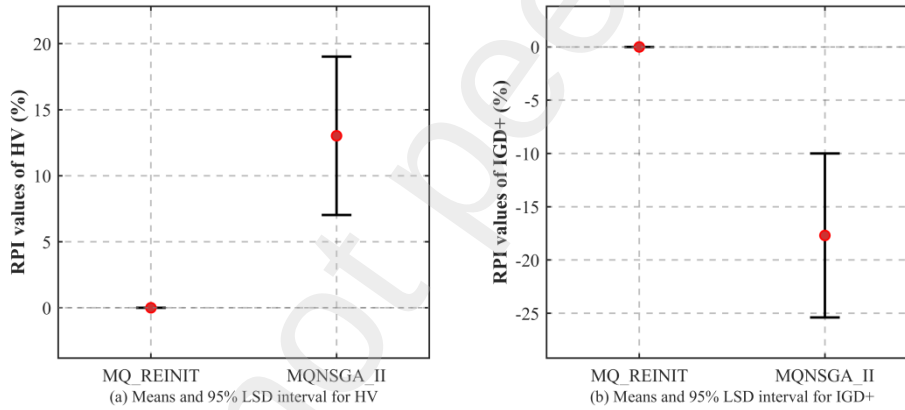


Fig. 13 Confidence intervals for MQNSGA-II and MQ_REINIT.

5.5. Comparisons with Efficient Algorithms

To thoroughly assess the performance of the proposed MQNSGA-II algorithm in addressing the SMO-RHFSP-BDST problem, this section conducts a comparison with four representative multi-objective optimization algorithms, including the classical NSGA-II (Deb et al.,2002), the improved algorithm Jaya (Pan et al.,2023), the decomposition-based modified MOEA/D (Wang et al.,2021), and the Q-learning integrated QVNS-NSGA-II (Li et al.,2023). For a fair and reliable evaluation, all algorithms are independently executed 15 times on 18 test instances of different scales under identical experimental conditions and settings.

Table 10 and Table 11 summarize the IGD⁺ and HV values for all five algorithms across the 18 test instances, with the best results highlighted in bold. Overall, MQNSGA-II consistently demonstrates stable advantages in both indicators. In terms of IGD⁺, MQNSGA-II achieves the best value in 14 out of 18 instances, reducing the IGD⁺ by 33.1%, 67.1%,

59.9%, and 28.5% compared to NSGA-II, Jaya, MOEA/D, and QVNS-NSGA-II, respectively. This indicates that the non-dominated solution set obtained by MQNSGA-II is more precise in approximating the reference Pareto front. Meanwhile, MQNSGA-II achieves the best HV value in 17 out of 18 instances, with improvements of 23.2%, 137.6%, 93.8%, and 17.4% over other algorithms, respectively. This demonstrates that MQNSGA-II is capable of achieving broader coverage in the objective space, providing a more diverse set of trade-off solutions.

Table 10 Comparison of IGD⁺ of algorithms.

Instance	IGD ⁺				
	NSGA II	Jaya	MOEA/D	QVNS NSGA II	MQNSGA-II
20-3-3	0.171487	0.37361	0.276106	0.170644	0.118714
20-3-4	0.193139	0.434109	0.161612	0.201879	0.103119
20-3-6	0.102435	0.221057	0.186072	0.124782	0.109728
20-5-3	0.151745	0.272545	0.363172	0.166177	0.116714
20-5-4	0.187959	0.397393	0.349639	0.251114	0.156414
20-5-6	0.255504	0.524928	0.461508	0.288064	0.180508
50-3-3	0.13209	0.43093	0.45591	0.09231	0.100535
50-3-4	0.147418	0.269668	0.262791	0.124421	0.130854
50-3-6	0.191899	0.411082	0.321865	0.184904	0.13592
50-5-3	0.2642	0.555418	0.374737	0.233142	0.133472
50-5-4	0.206801	0.42152	0.337524	0.176994	0.148818
50-5-6	0.312261	0.571974	0.352322	0.347495	0.135544
80-3-3	0.227081	0.334021	0.308366	0.183587	0.141942
80-3-4	0.196963	0.453859	0.310619	0.191591	0.147857
80-3-6	0.291553	0.51579	0.38865	0.253839	0.193271
80-5-3	0.250476	0.46839	0.367198	0.175061	0.159071
80-5-4	0.168318	0.383535	0.386075	0.085869	0.10327
80-5-6	0.174384	0.326113	0.379048	0.140597	0.108908
Avg	0.201429	0.409219	0.335734	0.188471	0.134703

Table 11 Comparison of the HV of the algorithms.

Instance	HV				
	NSGA II	Jaya	MOEA/D	QVNS NSGA II	MQNSGA-II
20-3-3	0.607174	0.326628	0.407735	0.624017	0.699433
20-3-4	0.567372	0.261207	0.519768	0.552024	0.722766
20-3-6	0.617367	0.413665	0.411631	0.599276	0.615225
20-5-3	0.532216	0.351128	0.2694	0.514166	0.61362
20-5-4	0.522329	0.256383	0.307108	0.434714	0.607736
20-5-6	0.538092	0.253042	0.296424	0.495554	0.63469
50-3-3	0.553426	0.236093	0.227793	0.620613	0.663781
50-3-4	0.487556	0.297452	0.351161	0.528839	0.535169
50-3-6	0.511434	0.251305	0.302508	0.541493	0.634421
50-5-3	0.422807	0.160787	0.308263	0.469007	0.642247
50-5-4	0.473691	0.208195	0.26625	0.50628	0.574231
50-5-6	0.438716	0.195426	0.33803	0.439832	0.682704
80-3-3	0.515612	0.370592	0.256283	0.571719	0.606469
80-3-4	0.552609	0.259624	0.409074	0.55491	0.643023
80-3-6	0.367136	0.167067	0.245616	0.420673	0.516771
80-5-3	0.44086	0.215416	0.333139	0.573298	0.601317
80-5-4	0.430608	0.199156	0.247518	0.541079	0.562322
80-5-6	0.429895	0.248684	0.228732	0.466418	0.543206
Avg	0.500494	0.259547	0.318135	0.525217	0.616618

To assess the statistical significance of the performance differences, NSGA-II is adopted as the baseline algorithm, and the RPI values for HV and IGD⁺ are analyzed using 95% LSD confidence intervals. As shown in Fig. 14, the confidence intervals of MQNSGA-II do not overlap with those of the other algorithms. This confirms the statistical significance of its performance advantage.

Furthermore, to analyze the trends of algorithm performance with increasing problem scale, the RPI values for each algorithm based on HV and IGD⁺ are categorized by problem size (20, 50, 80 jobs). The difference trends are shown in

Fig. 15 and Fig. 16. The shaded regions represent the 95% confidence intervals. It is evident that MQNSGA-II maintains superior performance over the comparison algorithms across all scales. Its advantage over the traditional multi-objective algorithms increases as the problem scale grows. This indicates that the proposed method exhibits good scalability and robustness. Compared to QVNS-NSGA-II, MQNSGA-II maintains a stable advantage across all problem sizes. This further demonstrates that the Meta-learning mechanism can effectively enhance the operator scheduling efficiency and improve search quality.

To visually illustrate the quality of the obtained Pareto solution sets, Fig. 17 and Fig. 18 present three-dimensional Pareto front scatter plots for a small scale instance (20-5-6) and a large scale instance (80-3-6), respectively. The purple ellipsoids represent the 95% confidence ellipsoids of MQNSGA-II. The results show that the Pareto solutions obtained by MQNSGA-II are closer to the lower region of the objective space. These solutions are also more evenly distributed and have wider coverage. This provides decision-makers with more trade-off solutions.

Overall, MQNSGA-II performs well on all test instances. It exhibits clear advantages in both Pareto front convergence and diversity. The statistical analyses further show that these advantages are significant, not just caused by random fluctuations. This means the proposed algorithm is effective and robust for solving complex multi-objective scheduling problems.

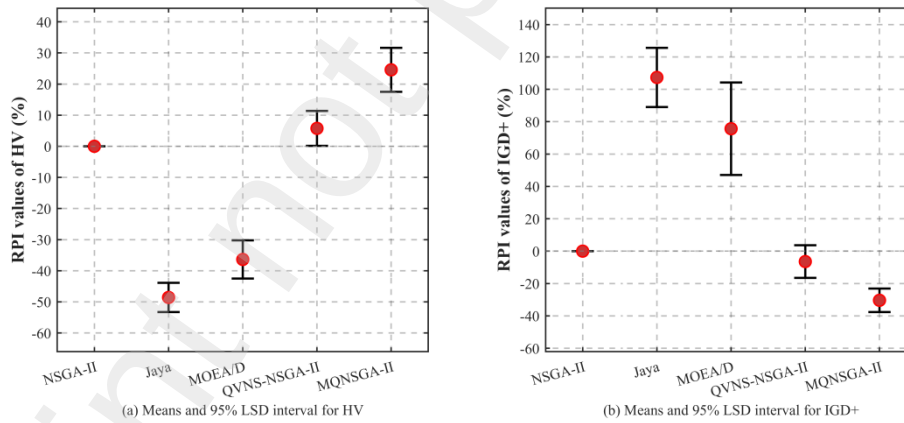


Fig. 14 Confidence intervals for algorithm comparison.

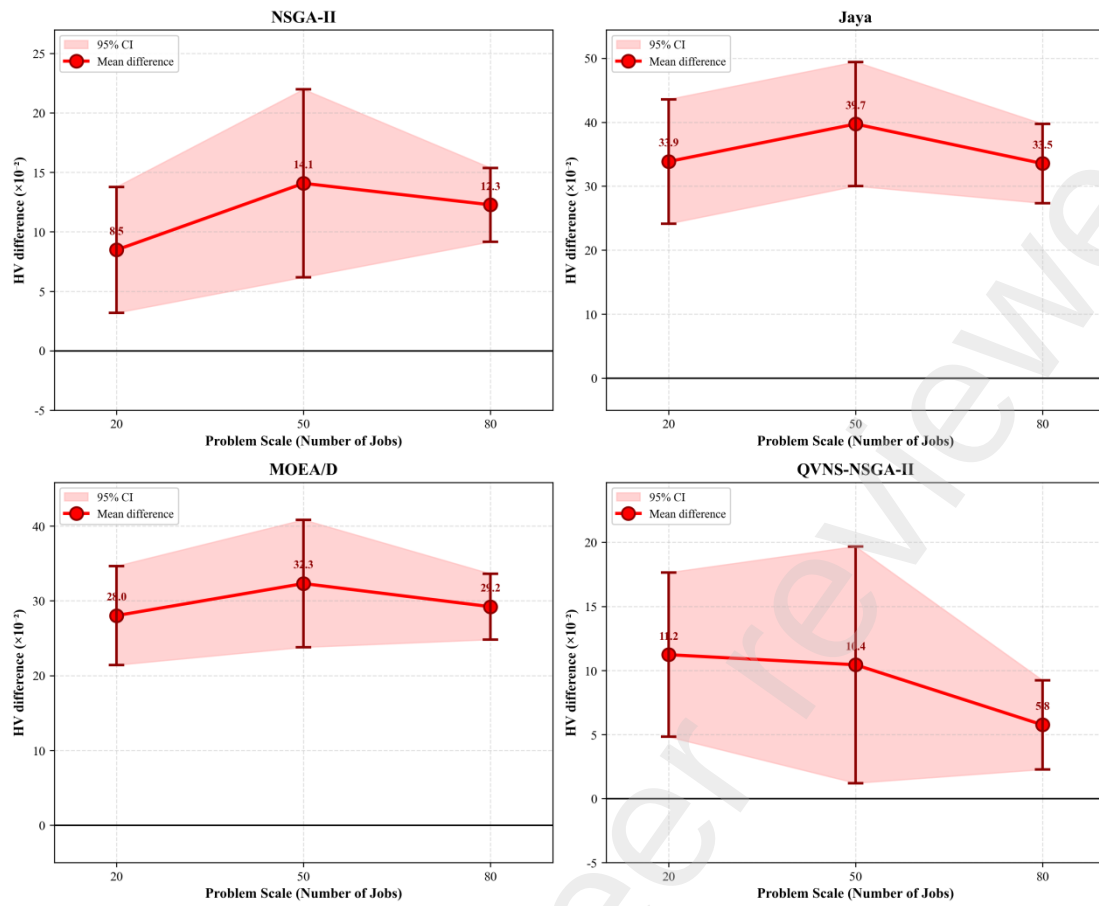


Fig. 15 Performance difference trend curve on HV.

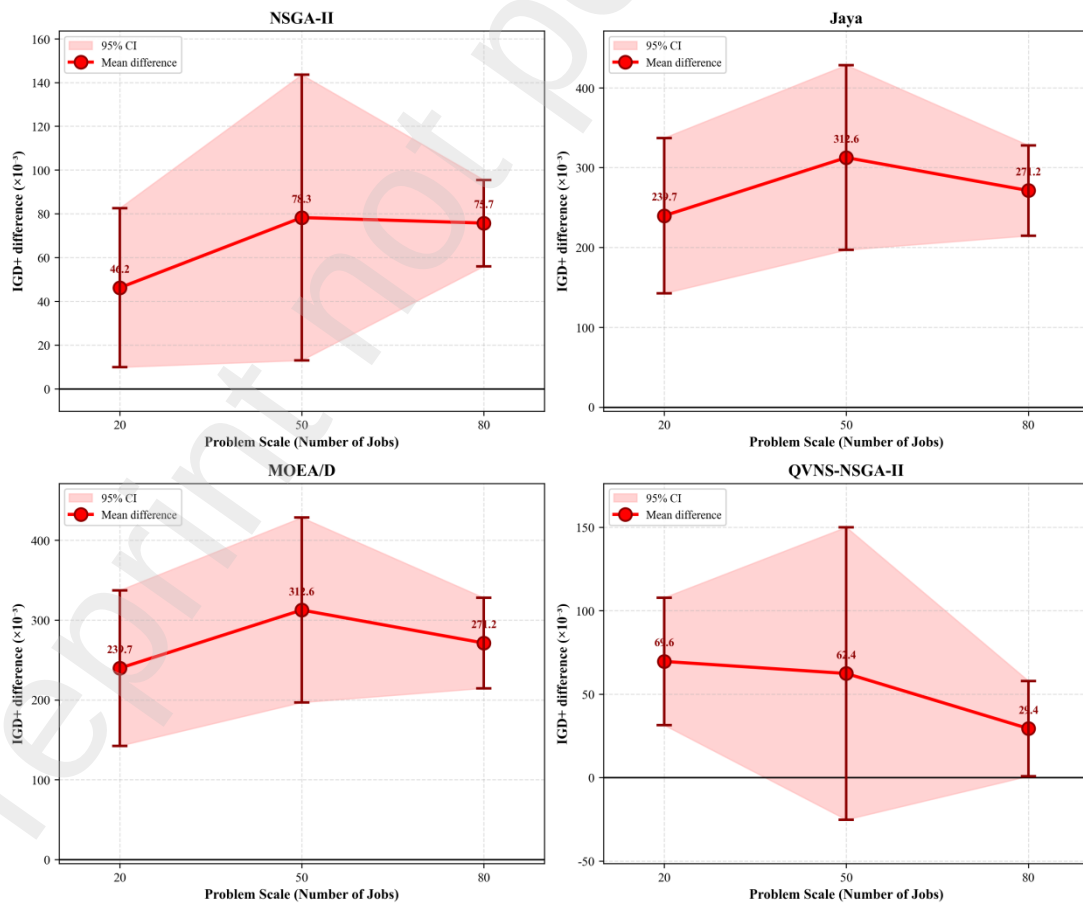


Fig. 16 Performance difference trend curve on IGD+.

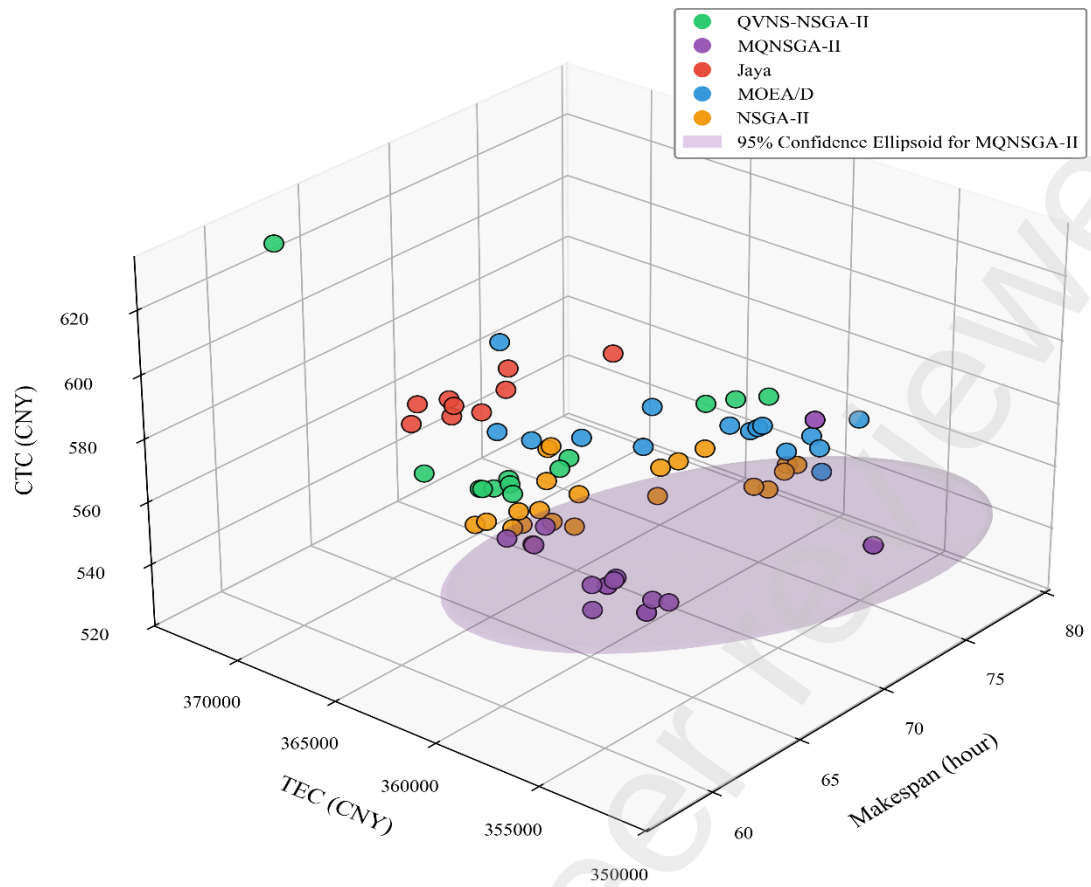


Fig. 17 3D Pareto front comparison for instance 20-5-6.

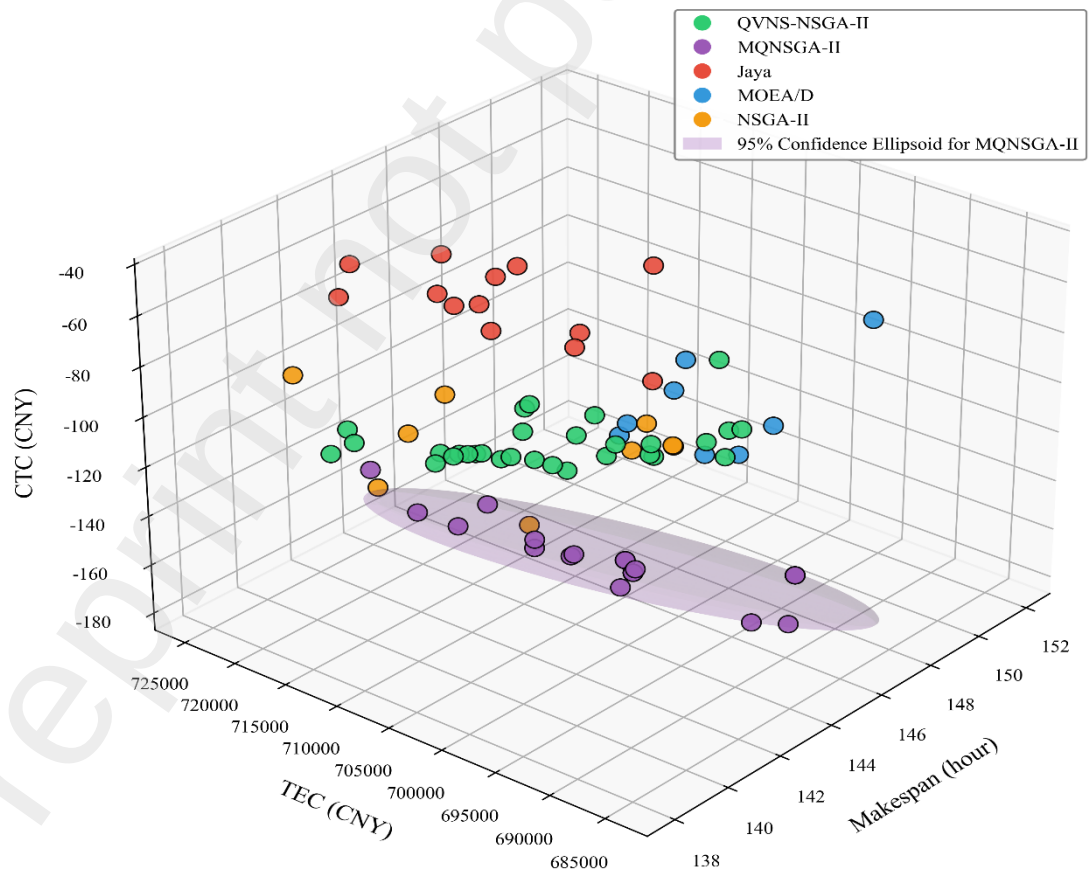


Fig. 18 3D Pareto front comparison for instance 80-3-6.

5.6. Experimental Analysis

Based on the results, the proposed MQNSGA-II shows stable advantages in solving the SMO-RHFSP-BDST. This is mainly due to the following. 1) The HHI mechanism builds a hybrid initial population. It combines NEH-based extreme solutions, random diversification, and Meta-training knowledge transfer. This accelerates early convergence and improves initial coverage. 2) The MQL-GVNS strategy, which leverages offline pre-trained Q-tables to guide the adaptive alternation between DC and VNS, achieving an effective exploration–exploitation balance and improving both convergence and distribution. 3) The HDCD and RR mechanisms, which strengthen solution-set diversity maintenance and stagnation recovery, respectively, reducing the risk of premature convergence. Fig. 19 presents a feasible Gantt chart for instance 50-3-3. The non-overlapping operations and the properly inserted setup activities further validate the feasibility of the proposed model and the effectiveness of the proposed algorithm.

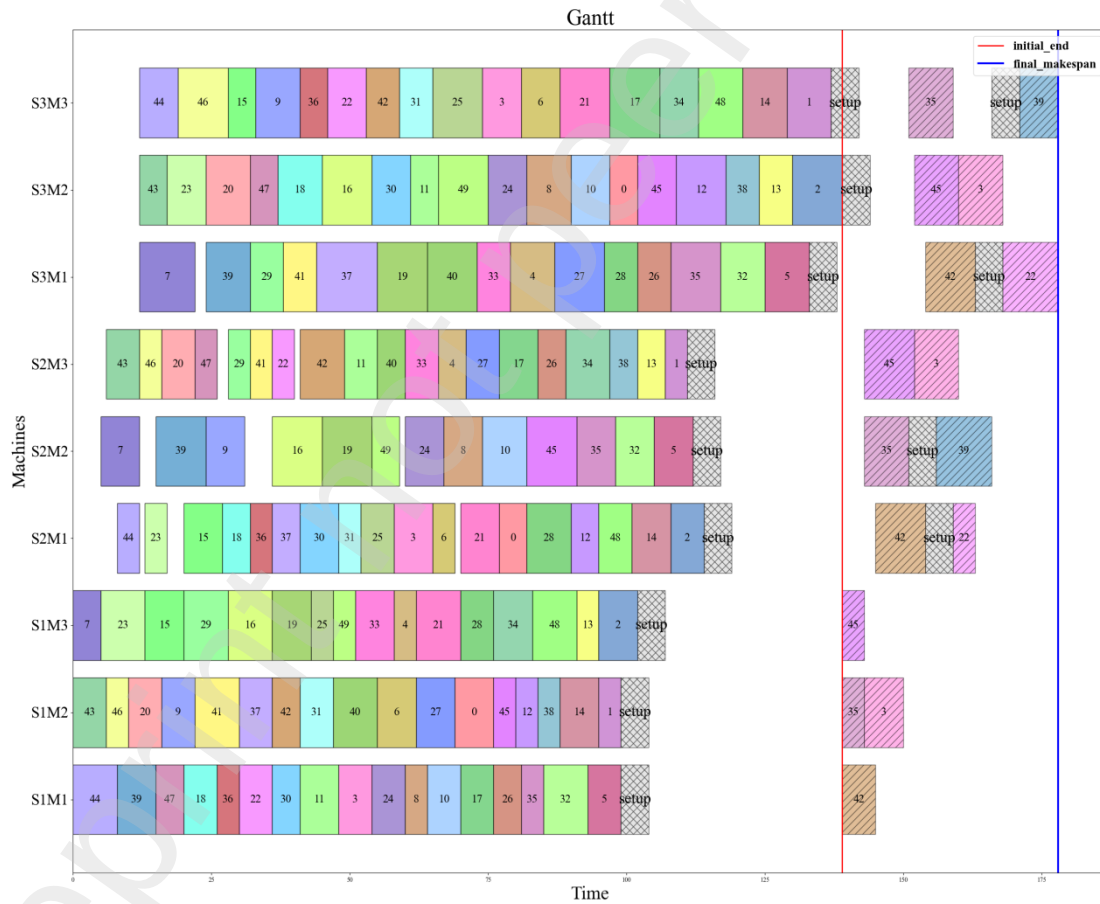


Fig. 19 Gantt of instance 50-3-3.

6. Conclusions

This paper investigated a sustainable multi-objective RHFSP with batch-dependent setup constraint. By jointly

considering the makespan, TEC, and CTC, a comprehensive multi-objective optimization model is formulated. Owing to the large problem scale, complex constraints, and pronounced conflicts among objectives, a MQL enhanced MOEA, termed MQNSGA-II, is proposed to effectively solve the problem. MQNSGA-II integrates Meta-learned adaptive neighborhood scheduling with diversity-handling and stagnation-handling mechanisms to robustly search for high quality Pareto trade-offs under coupled operational and sustainability constraints. Extensive numerical experiments demonstrate that the proposed MQNSGA-II consistently produces higher quality and more uniformly distributed Pareto-approximate fronts across test instances of different scales. With respect to both HV and IGD⁺, MQNSGA-II achieves better performance than several classical and state-of-the-art multi-objective algorithms. This shows the effectiveness of the proposed algorithmic components and their synergistic advantages.

Future research may extend the present work by incorporating stochastic processing times, dynamic electricity pricing mechanisms, and multiple rework paths. These extensions would further improve the applicability of the proposed model and solution approach in practical manufacturing systems.

References

- Chen, X., Li, J., Wang, Z., Chen, Q., Gao, K., & Pan, Q. (2025a). Optimizing Dynamic Flexible Job Shop Scheduling Using an Evolutionary Multitask Optimization Framework and Genetic Programming. *IEEE Transactions on Evolutionary Computation*, 29(5), 1502-1516. <https://doi.org/10.1109/TEVC.2025.3543770>
- Chen, X., Li, J., Wang, Z., Li, J., & Gao, K. (2025b). A genetic programming based cooperative evolutionary algorithm for flexible job shop with crane transportation and setup times. *Applied Soft Computing*, 169, 112614. <https://doi.org/10.1016/j.asoc.2024.112614>
- Costa, A. (2015). Hybrid genetic optimization for solving the batch-scheduling problem in a pharmaceutical industry. *Computers & Industrial Engineering*, 79, 130-147. <https://doi.org/10.1016/j.cie.2014.11.001>
- Cui, H., Li, X., & Gao, L. (2023). An improved multi-population genetic algorithm with a greedy job insertion inter-factory neighborhood structure for distributed heterogeneous hybrid flow shop scheduling problem. *Expert Systems with Applications*, 222, 119805. <https://doi.org/10.1016/j.eswa.2023.119805>
- Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002). A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2), 182-197. <https://doi.org/10.1109/4235.996017>
- Ding, J.-Y., Song, S., & Wu, C. (2016). Carbon-efficient scheduling of flow shops by multi-objective optimization. *European Journal of Operational Research*, 248(3), 758-771. <https://doi.org/10.1016/j.ejor.2015.05.019>
- Ding, J., Schulz, S., Shen, L., Buscher, U., & Lü, Z. (2021). Energy aware scheduling in flexible flow shops with hybrid particle swarm optimization. *Computers & Operations Research*, 125, 105088. <https://doi.org/10.1016/j.cor.2020.105088>
- Dong, J., & Ye, C. (2022). Green scheduling of distributed two-stage reentrant hybrid flow shop considering distributed energy resources and energy storage system. *Computers & Industrial Engineering*, 169, 108146. <https://doi.org/10.1016/j.cie.2022.108146>
- Drake, J. H., Kheiri, A., Özcan, E., & Burke, E. K. (2020). Recent advances in selection hyper-heuristics. *European Journal of Operational Research*, 285(2), 405-428. <https://doi.org/10.1016/j.ejor.2019.07.073>
- Fu, Y., Zhou, M., Guo, X., & Qi, L. (2020). Scheduling Dual-Objective Stochastic Hybrid Flow Shop With Deteriorating Jobs via Bi-Population Evolutionary Algorithm. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 50(12), 5037-5048. <https://doi.org/10.1109/TSMC.2019.2907575>
- Huang, J. D., Liu, J. J., Chen, Q. X., & Mao, N. (2016). Minimizing makespan in a two-stage flow shop with parallel batch-processing machines and re-entrant jobs. *Engineering Optimization*, 49(6), 1010-1023. <https://doi.org/10.1080/0305215X.2016.1231307>

- Hyeon-II, K., Yeo-Reum, K., & Dong-Ho, L. (2025). A genetic programming based reinforcement learning algorithm for dynamic hybrid flow shop scheduling with reworks under general queue time limits. *Computers & Industrial Engineering*, 203, 111062.<https://doi.org/10.1016/j.cie.2025.111062>
- Ishibuchi, H., Imada, R., Masuyama, N., & Nojima, Y. (2019). Comparison of Hypervolume, IGD and IGD+ from the Viewpoint of Optimal Distributions of Solutions. *International Conference on Evolutionary Multi-Criterion Optimization*, 332-345.https://doi.org/10.1007/978-3-030-12598-1_27
- Jiang, E., Wang, L., & Wang, J. (2021). Decomposition-based multi-objective optimization for energy-aware distributed hybrid flow shop scheduling with multiprocessor tasks. *Tsinghua Science and Technology*, 26(5), 646-663.<https://doi.org/10.26599/TST.2021.9010007>
- Karimi-Mamaghan, M., Mohammadi, M., Pasdeloup, B., & Meyer, P. (2022). Learning to select operators in meta-heuristics: An integration of Q-learning into the iterated greedy algorithm for the permutation flowshop. *European Journal of Operational Research*, 304(3), 1296-1330.<https://doi.org/10.1016/j.ejor.2022.03.054>
- Lei, Y., Gong, M., Zhang, J., Li, W., & Jiao, L. (2014). Resource allocation model and double-sphere crowding distance for evolutionary multi-objective optimization. *European Journal of Operational Research*, 234(1), 197-208.<https://doi.org/10.1016/j.ejor.2013.09.007>
- Li, C., Han, H., Wang, H., Liu, Y., Zhang, B., & Meng, L. (2025). Feature-driven double deep Q-network with iterated greedy for intelligent scheduling optimization in reentrant hybrid flow shops. *Engineering Applications of Artificial Intelligence*, 161, 112012.<https://doi.org/10.1016/j.engappai.2025.112012>
- Li, J., Li, J., Gao, K., & Duan, P. (2024). A Hybrid Graph-Based Imitation Learning Method for a Realistic Distributed Hybrid Flow Shop With Family Setup Time. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 54(12), 7291-7304.<https://doi.org/10.1109/TSMC.2024.3449413>
- Li, P., Xue, Q., Zhang, Z., Chen, J., & Zhou, D. (2023). Multi-objective energy-efficient hybrid flow shop scheduling using Q-learning and GVNS driven NSGA-II. *Computers & Operations Research*, 159, 106360.<https://doi.org/10.1016/j.cor.2023.106360>
- Lin, C.-C., Liu, W.-Y., & Chen, Y.-H. (2020). Considering stockers in reentrant hybrid flow shop scheduling with limited buffer capacity. *Computers & Industrial Engineering*, 139, 106154.<https://doi.org/10.1016/j.cie.2019.106154>
- Lin, C.-C., Peng, Y.-C., Chang, Y.-S., & Chang, C.-H. (2024). Reentrant hybrid flow shop scheduling with stockers in automated material handling systems using deep reinforcement learning. *Computers & Industrial Engineering*, 189, 109995.<https://doi.org/10.1016/j.cie.2024.109995>
- Lu, B., Gao, K., Ren, Y., Li, D., & Slowik, A. (2024). Combining meta-heuristics and Q-learning for scheduling lot-streaming hybrid flow shops with consistent sublots. *Swarm and Evolutionary Computation*, 91, 101731.<https://doi.org/10.1016/j.swevo.2024.101731>
- Lu, C., Liu, Q., Zhang, B., & Yin, L. (2022). A Pareto-based hybrid iterated greedy algorithm for energy-efficient scheduling of distributed hybrid flowshop. *Expert Systems with Applications*, 204, 117555.<https://doi.org/10.1016/j.eswa.2022.117555>
- Lu, S., Zhang, X., Kong, M., & Fathollahi-Fard, A. M. (2025). Two-stage hybrid flow shop scheduling with sequence-dependent setup times in semiconductor manufacturing: A customized variable neighborhood search. *Annals of Operations Research*, 1-36.<https://doi.org/10.1007/s10479-025-06541-8>
- Luo, H., Du, B., Huang, G. Q., Chen, H., & Li, X. (2013). Hybrid flow shop scheduling considering machine electricity consumption cost. *International Journal of Production Research*, 146(2), 423-439.<https://doi.org/10.1016/j.ijpe.2013.01.028>
- Mansouri, S. A., Aktas, E., & Besikci, U. (2016). Green scheduling of a two-machine flowshop: Trade-off between makespan and energy consumption. *European Journal of Operational Research*, 248(3), 772-788.<https://doi.org/10.1016/j.ejor.2015.08.064>
- Naderi, B., Ruiz, R., & Zandieh, M. (2010). Algorithms for a realistic variant of flowshop scheduling. *Computers & Operations Research*, 37(2), 236-246.<https://doi.org/10.1016/j.cor.2009.04.017>
- Neufeld, J. S., Schulz, S., & Buscher, U. (2023). A systematic review of multi-objective hybrid flow shop scheduling. *European Journal of Operational Research*, 309(1), 1-23.<https://doi.org/10.1016/j.ejor.2022.08.009>
- Pan, Y., Gao, K., Li, Z., & Wu, N. (2023). Solving Biobjective Distributed Flow-Shop Scheduling Problems With Lot-Streaming Using an Improved Jaya Algorithm. *IEEE Transactions on Cybernetics*, 53(6), 3818-3828.<https://doi.org/10.1109/TCYB.2022.3164165>
- Ren, W., Yan, Y., Hu, Y., & Guan, Y. (2021). Joint optimisation for dynamic flexible job-shop scheduling problem with transportation time and resource constraints. *International Journal of Production Research*, 60(18), 5675-5696.<https://doi.org/10.1080/00207543.2021.1968526>
- Ren, Y., Gao, K., Fu, Y., Sang, H., Li, D., & Luo, Z. (2023). A novel Q-learning based variable neighborhood iterative search algorithm for solving disassembly line scheduling problems. *Swarm and Evolutionary Computation*, 80, 101338.<https://doi.org/10.1016/j.swevo.2023.101338>
- Ribas, I., Leisten, R., & Framiñan, J. M. (2010). Review and classification of hybrid flow shop scheduling problems from a production system and a

- solutions procedure perspective. *Computers & Operations Research*, 37(8), 1439-1454.<https://doi.org/10.1016/j.cor.2009.11.001>
- Ruiz, R., & Vázquez-Rodríguez, J. A. (2010). The hybrid flow shop scheduling problem. *European Journal of Operational Research*, 205(1), 1-18.<https://doi.org/10.1016/j.ejor.2009.09.024>
- Shahvari, O., & Logendran, R. (2016). Hybrid flow shop batching and scheduling with a bi-criteria objective. *International Journal of Production Economics*, 179, 239-258.<https://doi.org/10.1016/j.ijpe.2016.06.005>
- Shao, S., Xu, G., Li, J., Liu, Z., & Jin, Z. (2025). A job assignment scheduling algorithm with variable sublots for lot-streaming flexible job shop problem based on NSGAIL. *Computers & Operations Research*, 173, 106866.<https://doi.org/10.1016/j.cor.2024.106866>
- Shao, Z., Shao, W., Chen, J., & Pi, D. (2024). MQL-MM: A Meta-Q-Learning-Based Multiobjective Metaheuristic for Energy-Efficient Distributed Fuzzy Hybrid Blocking Flow-Shop Scheduling Problem. *IEEE Transactions on Evolutionary Computation*, 29(4), 1183-1198.<https://doi.org/10.1109/TEVC.2024.3399314>
- Sharma, S., & Kumar, V. (2022). A Comprehensive Review on Multi-objective Optimization Techniques: Past, Present and Future. *Archives of Computational Methods in Engineering*, 29(7), 5605-5633.<https://doi.org/10.1007/s11831-022-09778-9>
- Song, H., Li, J., Du, Z., et al. (2025). A Q-learning driven multi-objective evolutionary algorithm for worker fatigue dual-resource-constrained distributed hybrid flow shop. *Computers & Operations Research*, 175, 106919.<https://doi.org/10.1016/j.cor.2024.106919>
- Tao, X.-R., Pan, Q.-K., & Gao, L. (2024). An Iterated Greedy Algorithm With Reinforcement Learning for Distributed Hybrid Flowshop Problems With Job Merging. *IEEE Transactions on Evolutionary Computation*, 29(3), 589-600.<https://doi.org/10.1109/TEVC.2024.3443874>
- Wang, G., Li, X., Gao, L., & Li, P. (2021). Energy-efficient distributed heterogeneous welding flow scheduling problem using a modified MOEA/D. *Swarm and Evolutionary Computation*, 62, 100858.<https://doi.org/10.1016/j.swevo.2021.100858>
- Wang, J.-J., & Wang, L. (2022). A Cooperative Memetic Algorithm With Learning-Based Agent for Energy-Aware Distributed Hybrid Flow-Shop Scheduling. *IEEE Transactions on Evolutionary Computation*, 26(3), 461-475.<https://doi.org/10.1109/TEVC.2021.3106168>
- Wang, Y.-J., Wang, G.-G., Tian, F.-M., Gong, D.-W., & Pedrycz, W. (2023). Solving energy-efficient fuzzy hybrid flow-shop scheduling problem at a variable machine speed using an extended NSGA-II. *Engineering Applications of Artificial Intelligence*, 121, 105977.<https://doi.org/10.1016/j.engappai.2023.105977>
- Wang, Y.-n., Li, J.-q., Yang, Z.-z., et al. (2026). An adaptive collaborative optimization algorithm for hybrid flow shop with group setup times and consistent sublots. *Applied Mathematical Modelling*, 150, 116354.<https://doi.org/10.1016/j.apm.2025.116354>
- Wang, Z., Li, J., Chen, X., Duan, P., & Li, J. (2025). Uncertain Interruptibility Multiobjective Flexible Job Shop via Deep Reinforcement Learning Based on Heterogeneous Graph Self-Attention. *IEEE Transactions on Neural Networks and Learning Systems*, 36(10), 18598-18612.<https://doi.org/10.1109/tnnls.2025.3578368>
- Wu, J., & Liu, Y. (2025). A modified multi-agent proximal policy optimization algorithm for multi-objective dynamic partial-re-entrant hybrid flow shop scheduling problem. *Engineering Applications of Artificial Intelligence*, 140, 109688.<https://doi.org/10.1016/j.engappai.2024.109688>
- Wu, X., & Cao, Z. (2022). An improved multi-objective evolutionary algorithm based on decomposition for solving re-entrant hybrid flow shop scheduling problem with batch processing machines. *Computers & Industrial Engineering*, 169, 108236.<https://doi.org/10.1016/j.cie.2022.108236>
- Wu, X., & Xie, Z. (2023). Improved MOEA/D with local search for solving multistage distributed reentrant hybrid flow shop scheduling problem. *Expert Systems with Applications*, 232, 120893.<https://doi.org/10.1016/j.eswa.2023.120893>
- Wu, X., Yan, X., & Wang, L. (2022). Optimizing job release and scheduling jointly in a reentrant hybrid flow shop. *Expert Systems with Applications*, 209, 118278.<https://doi.org/10.1016/j.eswa.2022.118278>
- Wu, X., Yuan, Q., & Wang, L. (2021). Multiobjective Differential Evolution Algorithm for Solving Robotic Cell Scheduling Problem With Batch-Processing Machines. *IEEE Transactions on Automation Science and Engineering*, 18(2), 757-775.<https://doi.org/10.1109/TASE.2020.2969469>
- Yan, X., & Wu, X. (2022). IMOEA/D to optimize job release problem for a reentrant hybrid flow shop. *Computers & Industrial Engineering*, 163, 107800.<https://doi.org/10.1016/j.cie.2021.107800>
- Yu, H., Gao, K., Wu, N., Zhou, M., Suganthan, P. N., & Wang, S. (2024). Scheduling Multiobjective Dynamic Surgery Problems via Q-Learning-Based Meta-Heuristics. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 54(6), 3321-3333.<https://doi.org/10.1109/TSMC.2024.3352522>
- Yue, C., Song, J., Liang, J., et al. (2025). A multimodal multiobjective evolutionary algorithm based on neighborhood and enhanced special crowding distance. *Knowledge-Based Systems*, 315, 113340.<https://doi.org/10.1016/j.knosys.2025.113340>

- Yunusoglu, P., & Topaloglu Yildiz, S. (2021). Constraint programming approach for multi-resource-constrained unrelated parallel machine scheduling problem with sequence-dependent setup times. *International Journal of Production Research*, 60(7), 2212-2229. <https://doi.org/10.1080/00207543.2021.1885068>
- Zhang, Y., Lei, D., & Wang, J. (2024a). An artificial bee colony with diversified operators for energy-efficient hybrid flow shop scheduling with batch processing machines. *Engineering Applications of Artificial Intelligence*, 136, 108930. <https://doi.org/10.1016/j.engappai.2024.108930>
- Zhang, Z., Shao, Z., Shao, W., Chen, J., & Pi, D. (2024b). MRLM: A meta-reinforcement learning-based metaheuristic for hybrid flow-shop scheduling problem with learning and forgetting effects. *Swarm and Evolutionary Computation*, 85, 101479. <https://doi.org/10.1016/j.swevo.2024.101479>
- Zheng, Q., Zhang, Y., Tian, H., & He, L. (2024). A cooperative adaptive genetic algorithm for reentrant hybrid flow shop scheduling with sequence-dependent setup time and limited buffers. *Complex & Intelligent Systems*, 10(1), 781-809. <https://doi.org/10.1007/s40747-023-01147-8>
- Zheng, Y., Liu, Y., & Luo, X. (2026). Multi-objective scheduling for a reentrant hybrid flow shop integrating preventive maintenance, machine aging effects, and low-carbon optimization. *Information Sciences*, 731, 122910. <https://doi.org/10.1016/j.ins.2025.122910>